

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



LÊ NHƯ ĐẠT - 52200160

**BÁO CÁO GIỮA KỲ
NHẬP MÔN HỌC MÁY**

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025

**TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN**



LÊ NHƯ ĐẠT - 52200160

BÁO CÁO GIỮA KỲ NHẬP MÔN HỌC MÁY

Người hướng dẫn
TS. Trần Lương Quốc Đại

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến thầy Đại vì đã tận tình hướng dẫn, truyền đạt kiến thức và dành thời gian quý báu hỗ trợ chúng em trong suốt quá trình thực hiện bài báo cáo này. Những góp ý và chỉ dẫn của thầy đã giúp chúng em hiểu sâu hơn về đề tài, hoàn thiện kỹ năng nghiên cứu và trình bày. Chúng em xin trân trọng cảm ơn thầy và kính chúc thầy luôn mạnh khỏe, thành công trong sự nghiệp trồng người.

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH

TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của TS Trần Lương Quốc Đại. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 2 tháng 3 năm 2025

Tác giả

(Ký tên và ghi rõ họ tên)

Đạt

Lê Như Đạt

TÓM TẮT

Trong phần Bài 1, em tập trung nghiên cứu và so sánh sáu phương pháp tối ưu hóa dựa trên Gradient Descent Algorithm (GDA), bao gồm Batch Gradient Descent, Stochastic Gradient Descent, Mini-batch Gradient Descent, Momentum, RMSprop và Adam. Đầu tiên, mỗi phương pháp được mô tả chi tiết về định nghĩa, ý nghĩa và cơ chế tối ưu hóa thông qua công thức toán học, cùng với các ví dụ minh họa và hình vẽ trực quan về đường hội tụ của hàm mất mát. Tiếp theo, em triển khai code minh họa từng thuật toán trên một tập dữ liệu đơn giản để so sánh tốc độ và ổn định của quá trình huấn luyện. Cuối cùng, các thuật toán GDA này được ứng dụng vào bài toán dự đoán giá nhà bằng mô hình hồi quy tuyến tính; kết quả được đánh giá bằng các chỉ số Mean Squared Error (MSE), Mean Absolute Error (MAE) và hệ số xác định R-square.

Trong phần Bài 2, em chọn một tập dữ liệu UCI chưa từng sử dụng trong lớp với cả biến số và biến chữ. Quy trình bắt đầu bằng việc làm sạch và khám phá dữ liệu: xử lý giá trị thiếu, ngoại lệ, sau đó trực quan hóa phân phối và mối quan hệ giữa các biến bằng biểu đồ histogram, boxplot và bar chart. Tiếp theo, các biến chữ được mã hóa (One-Hot Encoding hoặc Label Encoding), biến số được chuẩn hóa (StandardScaler hoặc MinMaxScaler), rồi chia dữ liệu thành tập Train/Test theo tỷ lệ 80/20. Em triển khai ít nhất năm thuật toán phân loại—Logistic Regression, Decision Tree, Random Forest, SVM và K-Nearest Neighbors—và đánh giá hiệu năng trên tập Test qua các chỉ số Accuracy, Precision, Recall, F1-score và ROC-AUC. Cuối cùng, khái niệm overfitting được giải thích cùng với các phương pháp khắc phục như regularization, pruning, cross-validation và ensemble. Em áp dụng ít nhất một kỹ thuật chống overfitting cho mỗi mô hình và so sánh kết quả trước và sau khi áp dụng để minh chứng hiệu quả của các giải pháp này.

MỤC LỤC

DANH MỤC HÌNH VẼ	8
CHƯƠNG 1. CÂU 1.....	10
1.1 Gradient Descent	11
1.2 Batch Gradient Descent.....	13
1.3 Stochastic Gradient Descent	15
1.4 Mini-Batch Gradient Descent	17
1.5 Momentum	18
1.6 RMSprop	21
1.5 Adam	25
1.7 Bài toán dự nhà đoán giá nhà	27
CHƯƠNG 2. CÂU 2.....	28
2.1 Các bước làm sạch data, visualization	29
2.1.1 Kiểm tra số lượng thiếu.....	29
2.1.2 Kiểm tra sự trùng lặp.....	29
2.1.2 Trực quan hóa	29
2.2 Chuẩn hóa dữ liệu	37
2.3 Trình bày và thực hiện các thuật toán machine learning	38
2.3.1 Logistic Regression.....	38
2.3.2 Softmax Regression	39
2.3.3 Decision Tree	41
2.3.4 Random Forest	43
2.3.5 Support Vector Machine	47
2.3.6 K-Nearest Neighbors.....	48
2.4 Phương pháp, độ đo, và đánh giá kết quả	50
2.4.1 Bài toán Regression.....	50
2.4.2 Bài toán Classification	51

2.5 Overfitting	54
2.5.1 Cơ sở lý thuyết và phương pháp	54
2.5.2 So sánh	57
TÀI LIỆU THAM KHẢO	60

DANH MỤC HÌNH VẼ

Hình 1.1: Hàm số 3D để mô phỏng các thuật toán tối ưu.....	10
Hình 1.2: Ví dụ tính tay của gradient descent.....	12
Hình 1.3: Gradient Descent.....	12
Hình 1.4: Ví dụ trực quan về Gradient Descent.....	12
Hình 1.5: Code minh họa thuật toán Gradient Descent	13
Hình 1.6: Batch Gradient Descent	14
Hình 1.7: Code minh họa thuật toán Batch Gradient Descent.....	15
Hình 1.8: Stochastic Gradient Descent (1).....	16
Hình 1.9: Stochastic Gradient Descent (2).....	16
Hình 1.10: Code minh họa của Stochastic Gradient Descent.....	17
Hình 1.11. Mini-Batch Gradient Descent	18
Hình 1.12: Code minh họa Mini-Batch Gradient Descent.....	18
Hình 1.13: Momentum	20
Hình 1.14: Code minh họa Momentum	20
Hình 1.15. Root Mean Square Propagation	24
Hình 1.16: Code minh họa RMSProp	24
Hình 1.17: Code minh họa Adam	27
Hình 1.18: So sánh kết quả hội tụ của các thuật toán tối ưu.....	28
Hình 1.19: Số lượng giá trị thiếu cho từng cột.....	29
Hình 2.1: Histogram của các cột numerical.....	31
Hình 2.2: Boxplot cho tất cả các cột numerical	32
Hình 2.3: tần suất các giá trị của các cột categorical	34
Hình 2.4: Ma trận tương quan giữa các biến numerical.....	35
Hình 2.5: Biểu đồ pairplot cho tất cả các cột numerical	36
Hình 2.6: Logistic Regression.....	39
Hình 2.7: metrics và confusion matrix Logistic Regression.....	39
Hình 2.7: Softmax Regression	40
Hình 2.8: Decision Tree	43

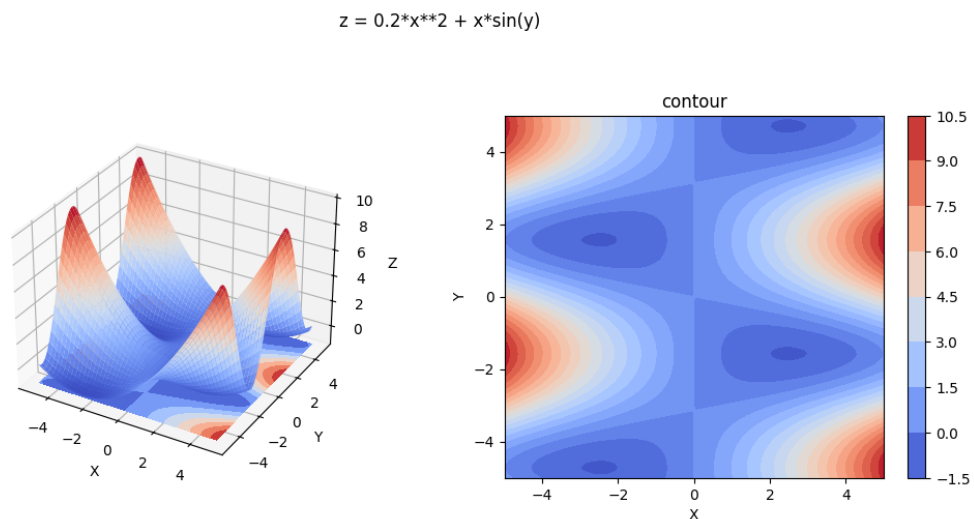
Hình 2.9: metrics và confusion matrix Decision Tree	43
Hình 2.10: Random Forest	46
Hình 2.11: metrics và confusion matrix Random Forest	46
Hình 2.12: Support Vector Machine	47
Hình 2.13: metrics và confusion matrix SVM	48
Hình 2.14: K-nearest neighbors	49
Hình 2.16: ROC curve.....	53
Hình 2.17: Confusion Matrix	54
Hình 2.18: Overfitting	57
Hình 2.19: So sánh mô hình.....	58

CHƯƠNG 1. CÂU 1

Đề bài yêu cầu ta phải trình bày các phương pháp để tối ưu hóa của thuật toán Gradient Descent (GD), nó bao gồm:

- Batch Gradient Descent (Batch GD)
- Stochastic Gradient Descent (SGD)
- Mini-Batch Gradient Descent
- Momentum
- RMSprop (Root Mean Square Propagation)
- Adam (Adaptive Moment Estimation)

Trong phần này: em sẽ sử dụng hàm số $z = \frac{1}{5}x^2 + x\sin(y)$ để minh họa cho các thuật toán này.



Hình 1.1: Hàm số 3D để mô phỏng các thuật toán tối ưu

1.1 Gradient Descent

Gradient Descent là một thuật toán tối ưu phổ biến trong học máy, dùng để tối thiểu hóa hàm mất mát (loss function) bằng cách cập nhật các trọng số (weights) theo hướng giảm dần của gradient. Công thức như sau:

Cơ chế tối ưu:

1. Khởi tạo: Chọn ngẫu nhiên một điểm bắt đầu θ_0 ($\theta_0 \in R$)
2. Tính gradient: Tính gradient của hàm Loss tại thời điểm hiện tại (t) theo công thức (dấu trừ thể hiện cho việc ta phải đi ngược lại hướng của gradient):

$$\theta_t = \theta_{t-1} - n \cdot \nabla L(\theta_{t-1}), \text{ trong đó}$$

$\nabla L(\theta_t)$: gradient Loss tại thời điểm t .

n : learning-rate (tốc độ học), một phần tỷ lệ của một bước dịch chuyển trọng số của mô hình.

θ_{t-1} : trọng số của mô hình

3. Cập nhật các tham số theo hướng giảm gradient.
4. Lặp lại cho đến khi hội tụ, hoặc đạt đến số bước lặp tối đa.

Ví dụ:

Giả sử ta có phương trình $y = w_0 + w_1 x_1 + w_2 x_2$

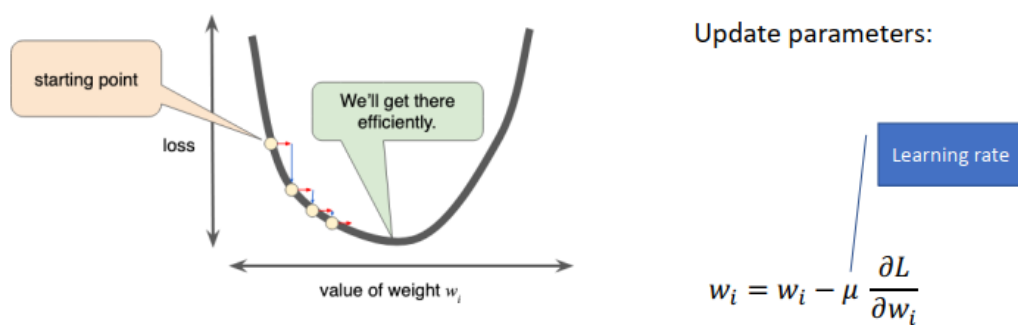
Với dữ liệu mẫu ($x_1 = 1, x_2 = 2, y = 3$), $n(\text{learning rate}) = 0.1$, $\theta_0 = (w_0, w_1, w_2) = (0, 0, 0)$. Ví dụ ở iteration 1:

- $y_{\text{hat}} = 0 + 0.1 + 0.2 = 0$ (y_{hat} là giá trị dự đoán)
- $L = \frac{1}{2} (y - y_{\text{hat}})^2 = \frac{1}{2} (3^2) = 4.5$
- $\frac{\partial L}{\partial w_0} = -3, \frac{\partial L}{\partial w_1} = -3, \frac{\partial L}{\partial w_2} = -6$
- $\theta_t = \theta_{t-1} - n \cdot \nabla L(\theta_{t-1}) = (0, 0, 0) - 0.1 \cdot (-3, -3, -6) = (0.3, 0.3, 0.6)$

Tiếp tục những iteration khác:

theta_old	L(theta)	theta(new)
0,0,0	4.5	0.3, 0.3, 0.6
0.3, 0.3, 0.6	0.72	0.42, 0.42, 0.84
0.42, 0.42, 0.84	0.1152	0.468, 0.468, 0.936

Hình 1.2: Ví dụ tính tay của gradient descent



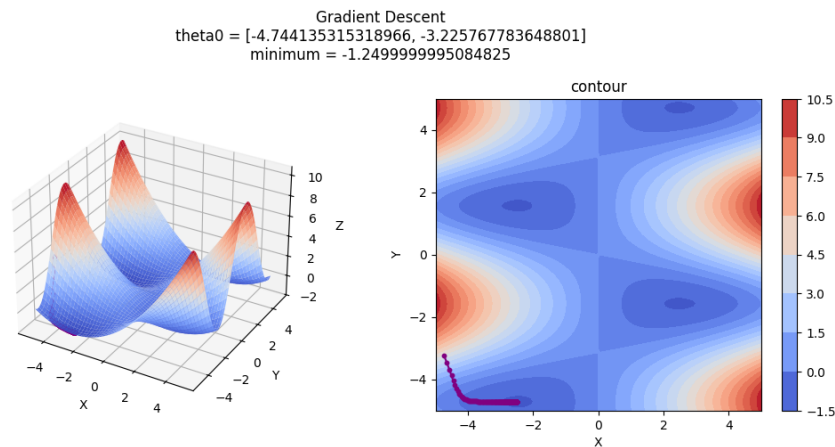
Hình 1.3: Gradient Descent

(Nguồn: internet)



Hình 1.4: Ví dụ trực quan về Gradient Descent

(Nguồn: internet)



Hình 1.5: Code minh họa thuật toán Gradient Descent

1.2 Batch Gradient Descent

Batch Gradient Descent (Batch GD) là một biến thể của Gradient Descent, nó sử dụng toàn bộ dữ liệu để tính trung bình gradient cho mỗi vòng lặp (epoch).

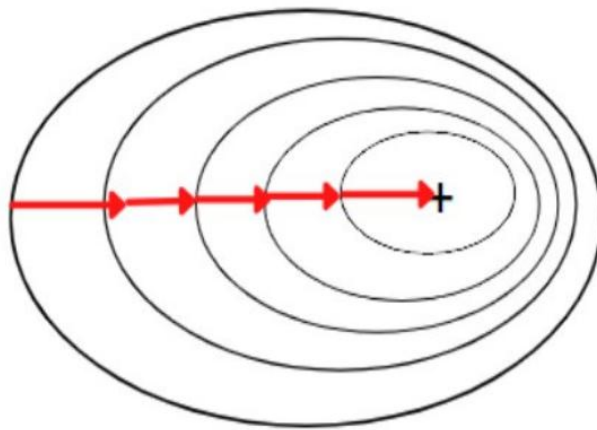
Ưu điểm của Batch Gradient Descent:

- **Độ ổn định cao:** Vì sử dụng toàn bộ dữ liệu trong mỗi vòng lặp, nên gradient được tính toán chính xác, giúp việc cập nhật tham số ổn định hơn và tránh được nhiễu do mẫu dữ liệu riêng lẻ gây ra.
- **Đảm bảo hội tụ:** Do tính toán gradient trung bình trên toàn bộ dataset, phương pháp này thường hội tụ theo hướng giảm thiểu hàm loss một cách mượt mà, đặc biệt đối với các bài toán có hàm loss liên tục và khả vi.
- **Dễ dàng triển khai:** Thuật toán khá đơn giản để cài đặt và phân tích, giúp việc nghiên cứu lý thuyết và tối ưu hóa trở nên dễ dàng hơn.

Nhược điểm của Batch Gradient Descent:

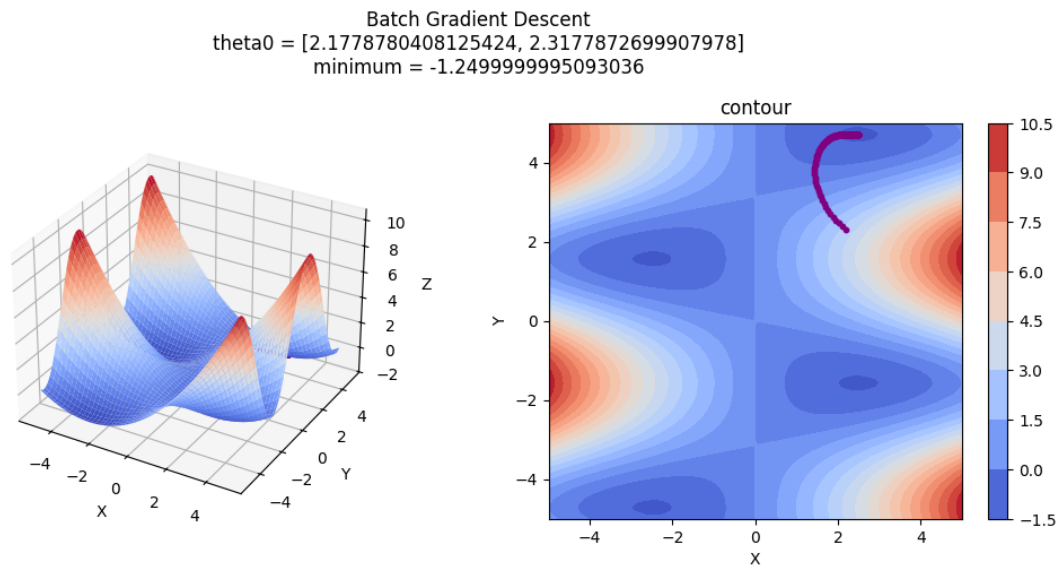
- **Tốn kém về tính toán và thời gian:** Với tập dữ liệu lớn, việc tính gradient trên toàn bộ dữ liệu cho mỗi lần cập nhật trở nên chậm chạp và không hiệu quả, đặc biệt khi số lượng mẫu rất lớn.
- **Yêu cầu bộ nhớ cao:** Phải tải toàn bộ dataset vào bộ nhớ để tính toán gradient, điều này có thể gây ra vấn đề khi làm việc với dữ liệu lớn.
- **Không phù hợp cho dữ liệu trực tuyến:** Phương pháp này không linh hoạt trong các trường hợp dữ liệu liên tục được cập nhật (streaming data), vì nó đòi hỏi phải có toàn bộ dữ liệu ngay từ đầu.

Batch Gradient Descent



Hình 1.6: Batch Gradient Descent

(Nguồn: internet)



Hình 1.7: Code minh họa thuật toán Batch Gradient Descent

1.3 Stochastic Gradient Descent

SGD là một biến thể của phương pháp gradient descent, trong đó thay vì tính toán gradient của hàm mất mát (loss function) trên toàn bộ tập dữ liệu, nó chỉ sử dụng một mẫu ngẫu nhiên tại mỗi bước cập nhật (sau mỗi epoch, chúng ta cần shuffle thứ tự của các dữ liệu để đảm bảo được tính ngẫu nhiên).

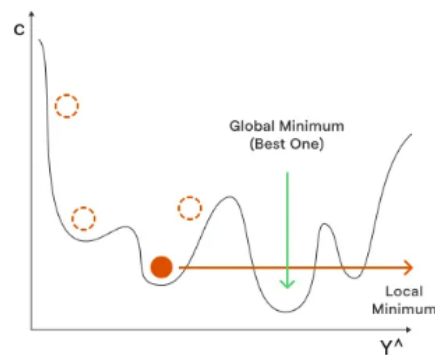
Ưu điểm:

- Hiệu quả tính toán: Vì mỗi bước chỉ cần một mẫu, SGD giảm đáng kể thời gian tính toán so với việc tính gradient trên toàn bộ dữ liệu, đặc biệt hữu ích với các tập dữ liệu lớn.
- Khả năng thoát khỏi cực trị địa phương: Do có tính chất ngẫu nhiên, SGD giúp thuật toán tránh bị mắc kẹt trong các cực trị địa phương, từ đó có thể tìm được cực tiểu toàn cục hoặc xung quanh cực tiểu tốt hơn.
- Dễ triển khai: SGD là một trong những thuật toán tối ưu hóa đơn giản nhất, dễ hiểu và dễ triển khai trong các bài toán học máy.

- Hiệu quả với dữ liệu liên tục

Nhược điểm: Do sử dụng từng mẫu riêng lẻ ngẫu nhiên, các trọng số được cập nhật của SGD thường không ổn định, dẫn đến dao động xung quanh giá trị tối ưu.

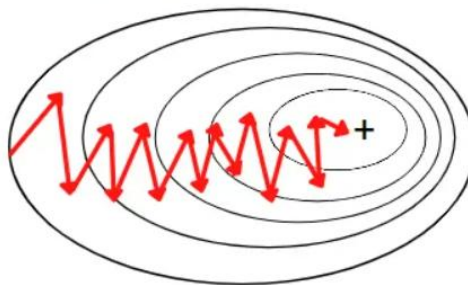
Stochastic Gradient Descent



Hình 1.8: Stochastic Gradient Descent (1)

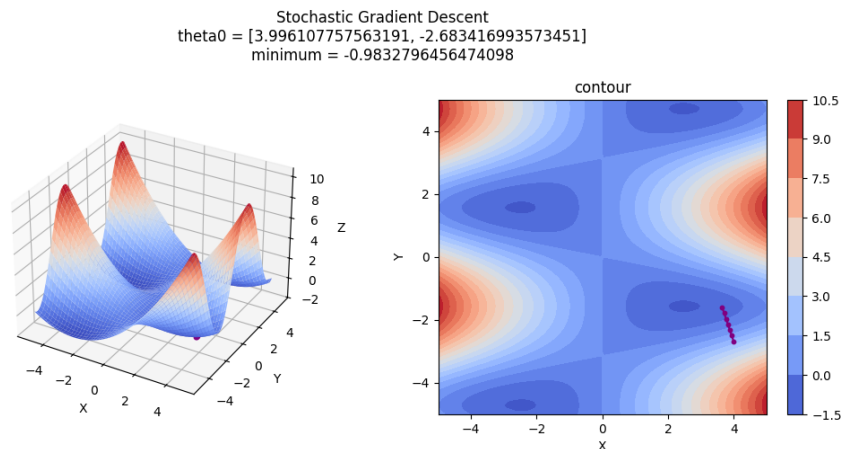
(Nguồn: internet)

Stochastic Gradient Descent



Hình 1.9: Stochastic Gradient Descent (2)

(Nguồn: internet)



Hình 1.10: Code minh họa của Stochastic Gradient Descent

1.4 Mini-Batch Gradient Descent

Mini-Batch Gradient Descent là một biến thể của Gradient Descent, trong đó tập dữ liệu được chia thành các "mini-batches" (tập con nhỏ). Thay vì cập nhật tham số sau mỗi mẫu (như trong Stochastic Gradient Descent) hoặc sau toàn bộ tập dữ liệu (như trong Batch Gradient Descent), mini-batch gradient descent cập nhật tham số sau mỗi mini-batch.

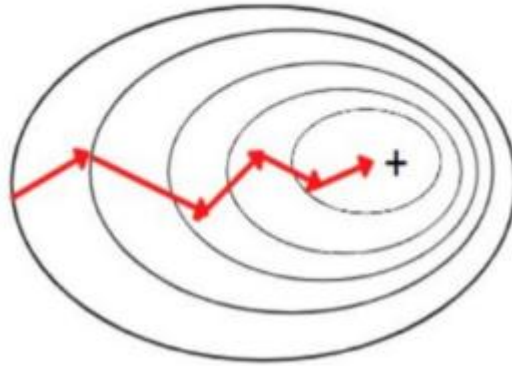
Ưu điểm:

- Tốc độ cập nhật nhanh hơn Batch GD, ổn định hơn SGD
- Tối ưu hóa hiệu năng, đặc biệt khi sử dụng GPU

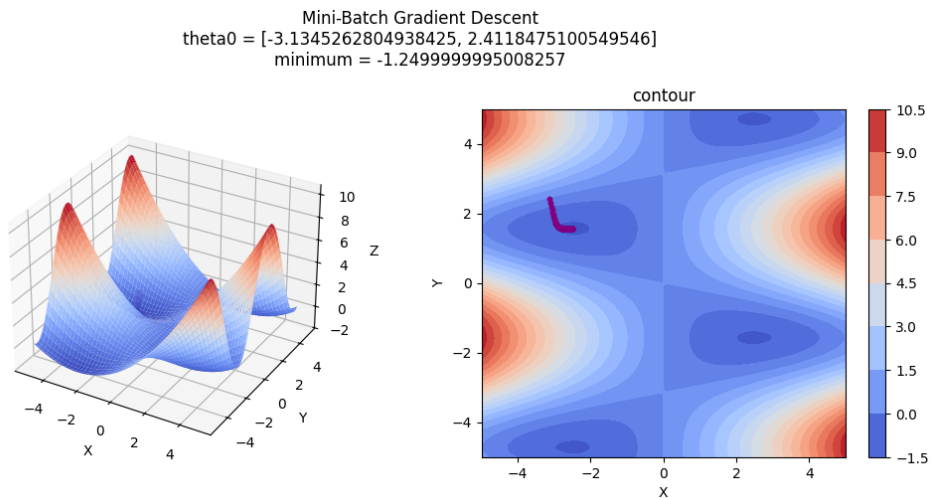
Nhược điểm:

- Phụ thuộc nhiều vào việc chọn kích thước của mini-batch.
- Nếu mini-batch quá nhỏ sẽ dẫn đến dao động lớn giống SGD
- Nếu mini-batch quá lớn sẽ dẫn đến tốn kém về hiệu quả tính toán và tốc độ cập nhật

Mini-Batch Gradient Descent



Hình 1.11. Mini-Batch Gradient Descent
(Nguồn internet)



Hình 1.12: Code minh họa Mini-Batch Gradient Descent

1.5 Momentum

Momentum là một phương pháp giúp điều chỉnh tốc độ cập nhật tham số của mô hình bằng cách thêm một thành phần vận tốc(velocity) vào quá trình cập nhật. Ý tưởng chính là sử dụng thông tin từ các bước cập nhật trước đó để làm cho quá trình tối ưu hóa trở nên mượt mà hơn.

Ý nghĩa:

- **Giảm dao động:** Momentum giúp giảm thiểu sự dao động trong quá trình cập nhật tham số, đặc biệt là trong các vùng có độ dốc thấp.
- **Tăng tốc độ hội tụ:** Bằng cách duy trì velocity, momentum có thể giúp mô hình hội tụ nhanh hơn đến cực tiểu của hàm mất mát.

Cơ chế hoạt động:

- Momentum lưu trữ một giá trị velocity cho mỗi tham số, được cập nhật theo thời gian.
- Tại mỗi bước cập nhật, giá trị velocity được tính toán dựa trên gradient hiện tại và giá trị động lượng trước đó.

Công thức:

$$v_t = \beta v_{t-1} - n \nabla L(\theta_{t-1})$$

$$\theta_t = \theta_{t-1} + v_{t-1}$$

β : hệ số Momentum, thường nằm trong khoảng (0, 1) (thông thường $\beta = 0.9$) điều chỉnh mức độ ảnh hưởng của các bước trước đó.

Ưu điểm:

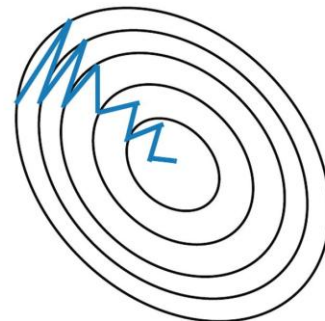
- **Tăng tốc độ hội tụ:** Nhờ tích lũy thông tin của các bước cập nhật trước, thuật toán có thể nhanh chóng di chuyển theo hướng của cực tiểu.
- **Giảm dao động:** Momentum giúp hạn chế sự dao động không cần thiết, đặc biệt khi gradient có sự thay đổi lớn giữa các bước cập nhật.

Nhược điểm:

- **Cần điều chỉnh tham số:** Hệ số momentum β cần được điều chỉnh cẩn thận; giá trị quá cao có thể làm quá mức “đẩy” tham số, dẫn đến vượt qua cực tiểu.
- **Độ phức tạp:** Mặc dù không quá phức tạp, nhưng việc lưu trữ và cập nhật thêm biến velocity làm tăng một chút độ phức tạp trong thuật toán.



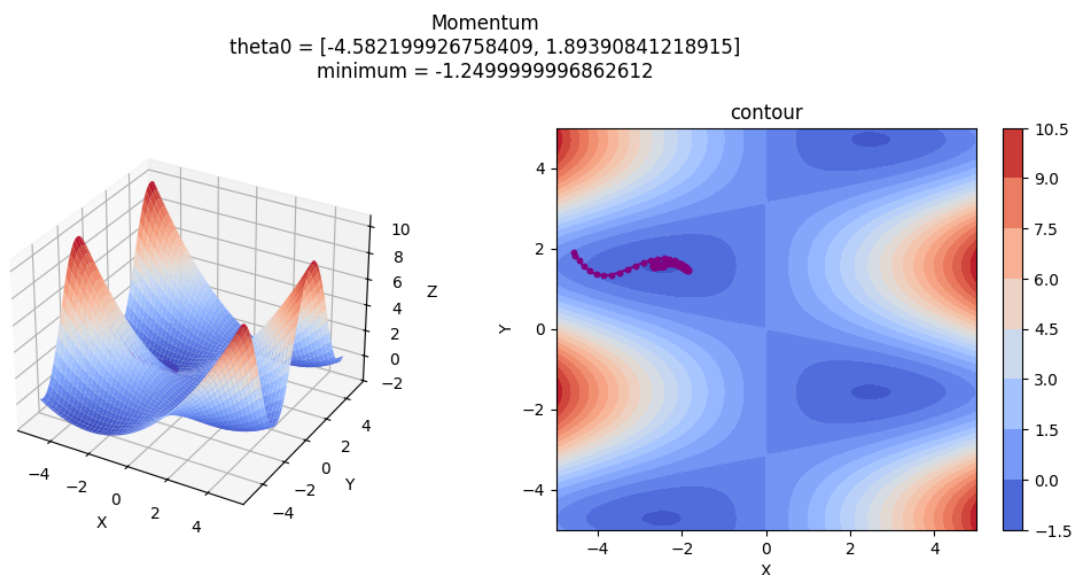
Stochastic Gradient
Descent **without**
Momentum



Stochastic Gradient
Descent **with**
Momentum

Hình 1.13: Momentum

(Nguồn: internet)



Hình 1.14: Code minh họa Momentum

1.6 RMSprop

RMSprop (Root Mean Square Propagation) là một thuật toán tối ưu hóa gradient descent được thiết kế để giải quyết vấn đề của tốc độ học không đồng nhất giữa các trọng số trong quá trình huấn luyện mô hình học máy. Thuật toán này được giới thiệu bởi Geoffrey Hinton và hướng đến việc điều chỉnh tốc độ học (learning rate) cho từng tham số dựa trên lịch sử các gradient của chúng.

Ý nghĩa:

- Điều chỉnh tốc độ học linh hoạt: RMSprop tự động điều chỉnh tốc độ học cho mỗi tham số dựa trên mức độ dao động của gradient. Điều này giúp tránh việc cập nhật quá lớn khi gradient có giá trị cao và quá nhỏ khi gradient có giá trị thấp.
- Ổn định quá trình học: Bằng cách sử dụng trung bình lũy thừa của các bình phương gradient, RMSprop giúp ổn định quá trình tối ưu, đặc biệt trong các bài toán có hàm mất mát (loss function) có nhiều cực tiểu cục bộ hoặc cực trị phẳng.
- Phù hợp với bài toán không ổn định: RMSprop thường được áp dụng hiệu quả cho các bài toán xử lý chuỗi, mạng nơ-ron hồi tiếp (RNN) và các mô hình phức tạp khác, nơi mà gradient có thể dao động mạnh.

Cơ chế tối ưu:

- **Theo dõi lịch sử gradient:** RMSprop duy trì một trung bình lũy thừa của các bình phương gradient (thường ký hiệu là $E[g^2]_t$). Điều này cho phép thuật toán “nhìn thấy” sự thay đổi của gradient theo thời gian.
- **Điều chỉnh tốc độ học:** Mỗi tham số có tốc độ học riêng được điều chỉnh theo giá trị trung bình của bình phương gradient. Nếu giá trị gradient lớn,

tốc độ học sẽ được giảm xuống để đảm bảo sự ổn định trong cập nhật trọng số.

- **Cập nhật tham số:** Các trọng số được cập nhật theo công thức, sử dụng tỉ lệ học đã được điều chỉnh nhằm tối ưu hóa hàm mất mát.

Công thức:

$$g_t = \nabla L(\theta_t)$$

$$E[g^2]_t = pE[g_{t-1}^2] + (1 - p)g_{t-1}^2$$

$$\theta_t = \theta_{t-1} - \frac{n}{\sqrt{E[g_{t-1}^2] + \varepsilon}} \cdot g_{t-1}$$

Trong đó:

g_t : là gradient tại bước t

$E[g^2]_t$: là trung bình lũy thừa của bình phương gradient tại thời điểm t

p (hoặc *decay*): hệ số suy giảm (thường khoảng 0.9)

n : learning rate ban đầu

ε : một hằng số nhỏ để đảm bảo không chia cho 0 (thường khoảng 10^{-8})

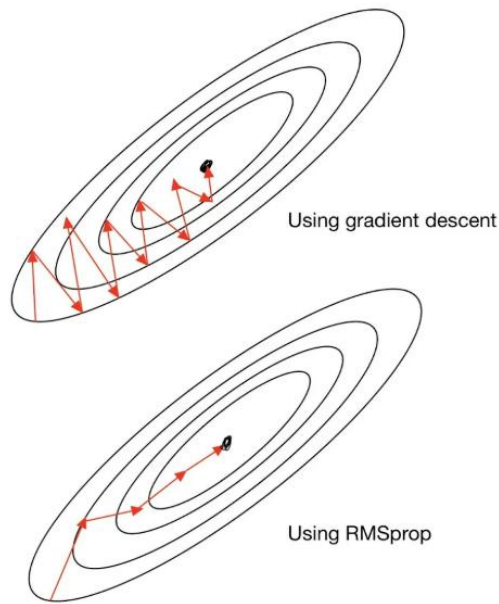
Ưu điểm:

- **Điều chỉnh tốc độ học tự động:** RMSprop điều chỉnh tốc độ học cho từng tham số dựa trên trung bình lũy thừa của bình phương gradient, giúp cải thiện quá trình hội tụ.

- **Ổn định trong quá trình học:** Nhờ cơ chế sử dụng trung bình lũy thừa của các gradient, RMSprop giảm bớt tác động của các gradient quá lớn, giúp quá trình cập nhật trọng số ổn định hơn.
- **Phù hợp với bài toán phức tạp:** Thuật toán được áp dụng hiệu quả trong các mô hình có cấu trúc phức tạp, đặc biệt là trong các bài toán liên quan đến dữ liệu chuỗi, mạng nơ-ron hồi tiếp (RNN) và các bài toán có hàm mất mát không ổn định.

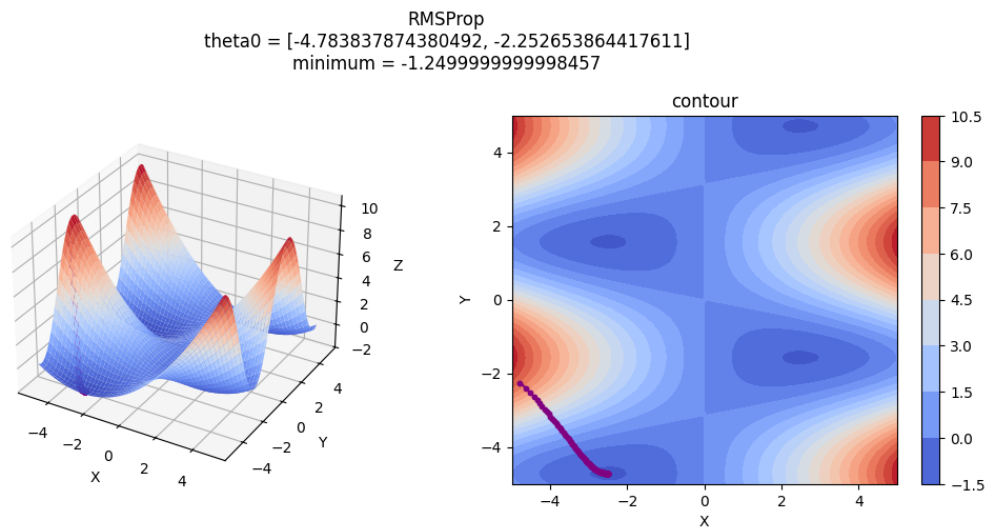
Nhược điểm:

- **Cần tinh chỉnh tham số:** RMSprop yêu cầu lựa chọn cẩn thận các tham số như tốc độ học ban đầu η , hệ số decay ρ và hằng số ϵ để đạt hiệu quả tối ưu, và không phải lúc nào cũng dễ dàng tìm được giá trị phù hợp.
- **Khả năng hội tụ không luôn tốt:** Trong một số trường hợp, RMSprop có thể không hội tụ tốt bằng các thuật toán khác như Adam, đặc biệt là khi dữ liệu hoặc hàm mất mát có đặc điểm phức tạp.
- **Thiếu cơ chế “nhớ” lâu dài:** Vì chỉ dựa vào trung bình lũy thừa của các bình phương gradient ngắn hạn, nên thuật toán có thể không lưu giữ đầy đủ thông tin của các gradient quá khứ, ảnh hưởng đến khả năng điều chỉnh tốc độ học khi cần thiết.



Hình 1.15. Root Mean Square Propagation

(Nguồn: internet)



Hình 1.16: Code minh họa RMSProp

1.5 Adam

Adam (Adaptive Moment Estimation) là thuật toán tối ưu hóa dùng để cập nhật trọng số trong quá trình huấn luyện mạng nơ-ron. Nó kết hợp ý tưởng từ Momentum (sử dụng trung bình của gradient) và RMSprop (điều chỉnh tốc độ học dựa trên trung bình bình phương gradient) để điều chỉnh tốc độ học một cách tự động cho từng tham số.

Ý nghĩa:

- **Tốc độ hội tụ nhanh:** Adam giúp mô hình hội tụ nhanh hơn so với các thuật toán tối ưu truyền thống nhờ việc sử dụng động lượng và điều chỉnh tốc độ học riêng cho từng tham số.
- **Ổn định và chính xác:** Việc kết hợp thông tin của gradient trung bình (động lượng) và trung bình bình phương gradient giúp giảm thiểu dao động trong quá trình cập nhật trọng số, dẫn đến sự ổn định cao hơn.
- **Tự động điều chỉnh tốc độ học:** Không cần quá nhiều thao tác tinh chỉnh tốc độ học vì Adam tự động điều chỉnh nhờ cơ chế thích nghi với đặc tính của gradient tại mỗi bước cập nhật.

Cơ chế tối ưu:

Adam hoạt động dựa trên hai "khoảnh khắc" của gradient:

- **Khoảnh khắc bậc nhất (m_t):** Là trung bình có trọng số của các gradient trước đó (tương tự như động lượng).
- **Khoảnh khắc bậc hai (v_t):** Là trung bình có trọng số của bình phương gradient (tương tự như cơ chế của RMSprop).

Ngoài ra, do các ước lượng này ban đầu có xu hướng lệch về 0, Adam sử dụng kỹ thuật điều chỉnh độ chệch (bias correction) để có ước lượng chính xác hơn trong các bước cập nhật ban đầu.

Công thức toán học:

1. Cập nhật khoảng khắc bậc nhất

$$m_t = \beta_1 \cdot m_{t-1} - 1 + (1 - \beta_1) \cdot g_t$$

2. Cập nhật khoảng khắc bậc 2

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

3. Điều chỉnh độ chệch

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

4. Cập nhật tham số:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$

Trong đó:

- g_t là gradient tại thời điểm t
- m_t là ước lượng của khoảng khắc bậc nhất tại thời điểm t
- v_t là ước lượng của khoảng khắc bậc hai tại thời điểm t
- β_1 và β_2 là các hệ số giảm dần (thông thường $\beta_1 \approx 0.9$ và $\beta_2 \approx 0.999$)

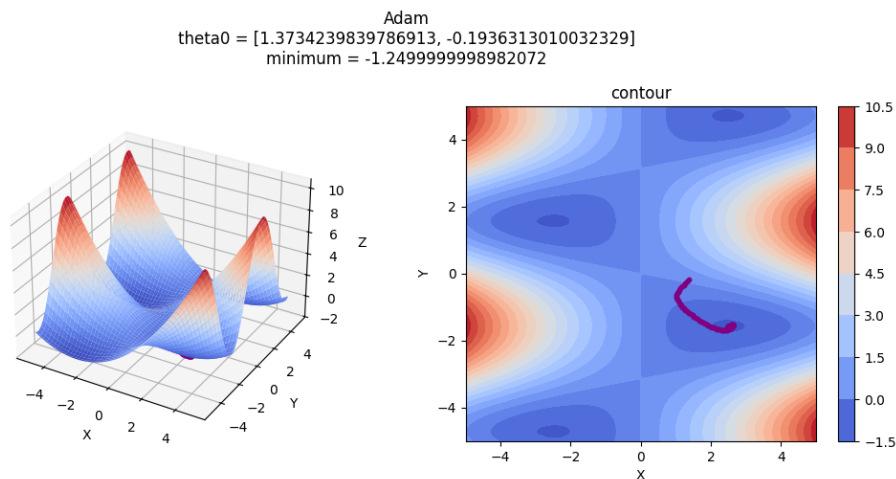
- η là tốc độ học (learning rate)
- ϵ là hằng số nhỏ để tránh chia cho 0 (thường khoảng 10^{-8})

Ưu điểm:

- Tích hợp cả thông tin của gradient và bình phương gradient giúp tối ưu hóa quá trình huấn luyện.
- Tốc độ hội tụ nhanh và thường ổn định hơn so với các phương pháp tối ưu khác.
- Ít đòi hỏi tinh chỉnh tốc độ học ban đầu, phù hợp với nhiều loại bài toán.

Nhược điểm:

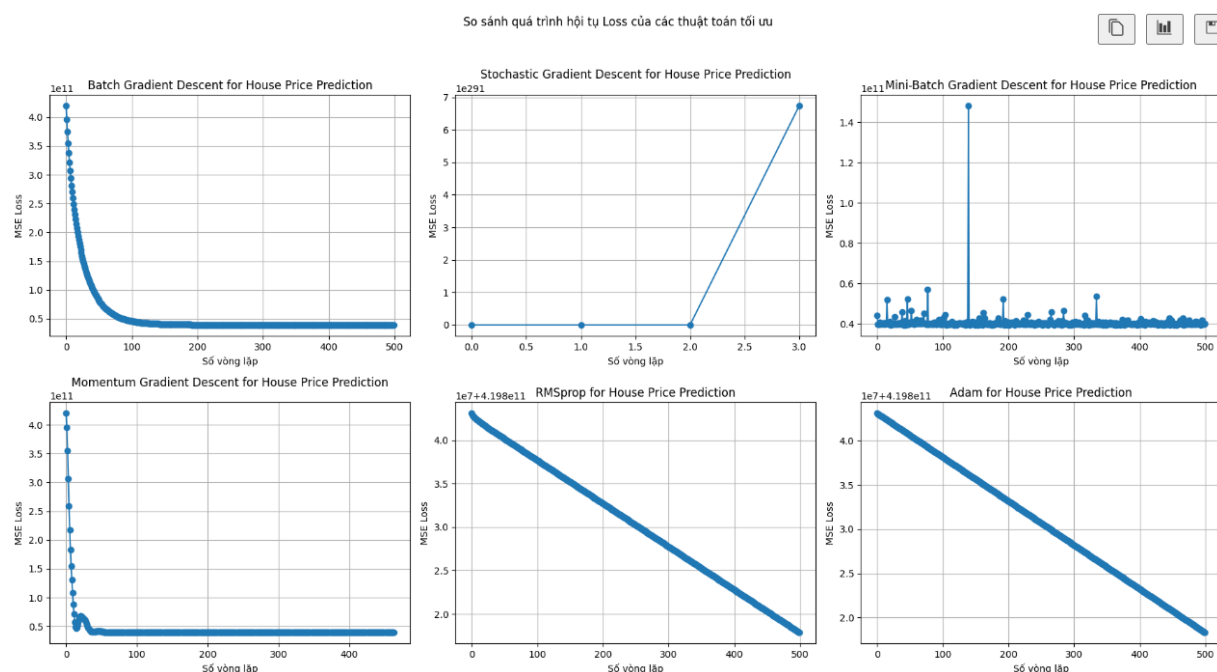
- Trong một số trường hợp, Adam có thể tạo ra các điểm cực trị giả (saddle points) hoặc hội tụ đến điểm cực trị cục bộ không tối ưu.
- Cần sử dụng thêm các kỹ thuật điều chỉnh (như decay tốc độ học) cho những bài toán rất phức tạp.



Hình 1.17: Code minh họa Adam

1.7 Bài toán dự nhà đoán giá nhà

Phần này em sẽ sử dụng bộ dữ liệu “house_prices.csv” từ kaggle để hiện thực hóa bài toán dự đoán giá nhà (link: [house_prices](#)), bộ dữ liệu này gồm có 21613 hàng và 19 cột, cột price là cột target. Em đã tự định nghĩa lại các lớp của các thuật toán Batch GD, Stochastic GD, Mini-Batch GD, Momentum, RMSProp, Adam cho phần dự đoán giá nhà.



Hình 1.18: So sánh kết quả hội tụ của các thuật toán tối ưu

Các thông số khác như MSE, RMSE, MAE, R-square, thời gian huấn luyện, loss em đã hiện thực ở trong phần code cau1.ipynb (src/cau1.ipynb).

CHƯƠNG 2. CÂU 2

Trong phần này, bộ dữ liệu em chọn trên UCI([UCI dataset](#)) là bộ dữ liệu được sử dụng để ước lượng mức độ béo phì của các cá nhân dựa trên thói quen ăn uống và tình trạng thể chất (ObesityDataSet_raw_and_data_synthetic.csv). Bộ dữ liệu này bao gồm

dữ liệu của người dân từ Mexico, Peru, và Colombia. Bộ dữ liệu có 2111 dòng và 17 cột, các feature đều đa dạng bao gồm dạng số và chữ, cột NObesity(Mức độ béo phì) là cột target.

2.1 Các bước làm sạch data, visualization

2.1.1 Kiểm tra số lượng thiếu

Em sẽ dùng hàm `isnull()` và hàm `sum()` để kiểm tra số lượng giá trị thiếu cho từng cột. Kết quả trả về là 0 cho tất cả các cột

```
----- Số lượng giá trị thiếu cho từng cột -----  
  
Gender                                0  
Age                                  0  
Height                              0  
Weight                              0  
family_history_with_overweight      0  
FAVC                                0  
FCVC                                0  
NCP                                  0  
CAEC                                0  
SMOKE                               0  
CH2O                                 0  
SCC                                  0  
FAF                                  0  
TUE                                  0  
CALC                                 0  
MTRANS                               0  
NObesidad                           0  
dtype: int64
```

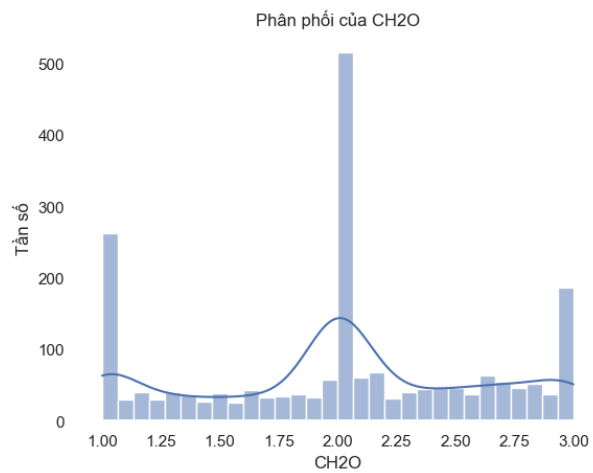
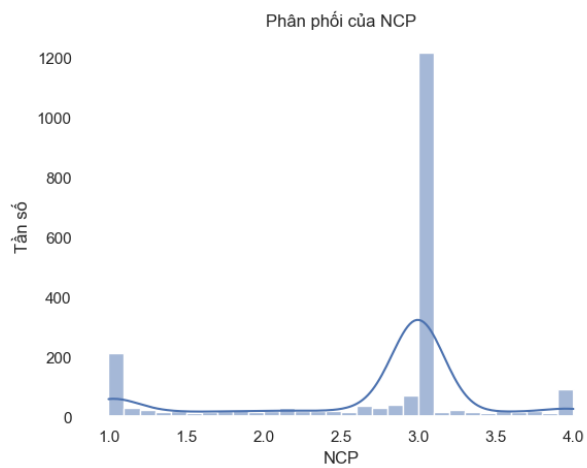
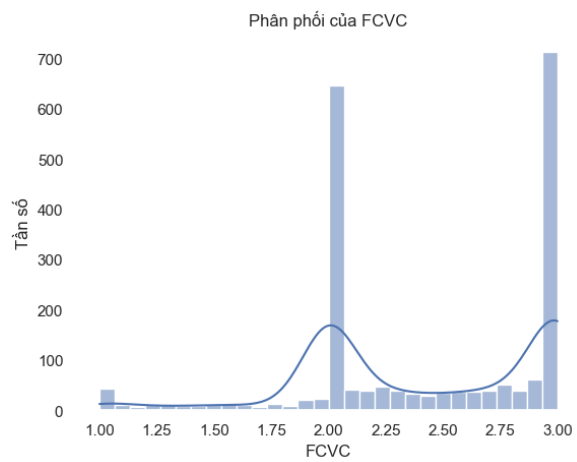
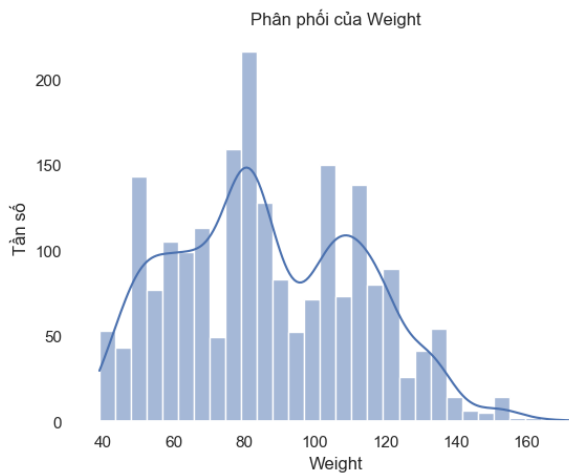
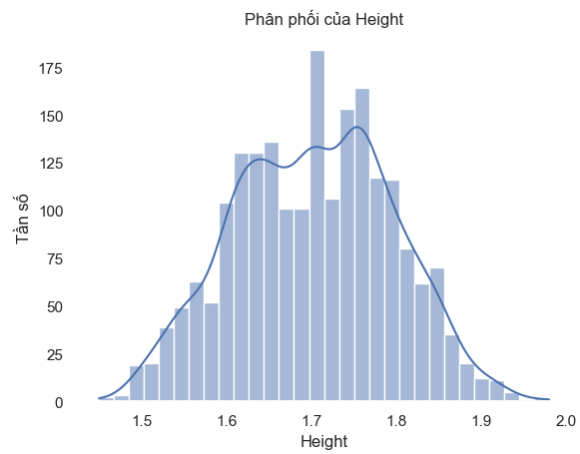
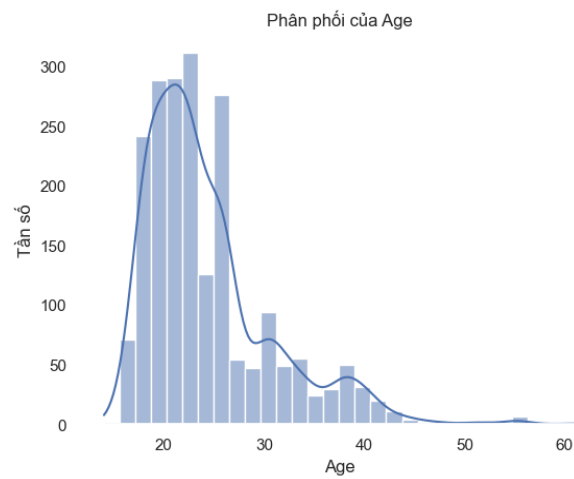
Hình 1.19: Số lượng giá trị thiếu cho từng cột

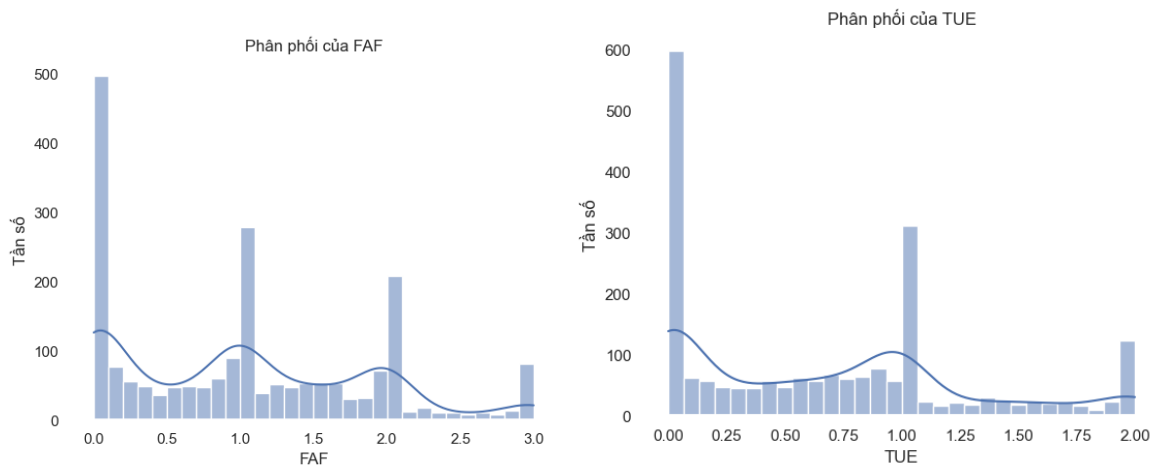
2.1.2 Kiểm tra sự trùng lặp

Em sẽ dùng hàm `duplicate()` và hàm `sum()` để kiểm tra số lượng giá trị trùng lặp, kết quả trả về là 24 dòng trùng lặp. Sau đó em sẽ dùng hàm `drop_duplicates()` để xóa những dòng bị trùng.

2.1.2 Trực quan hóa

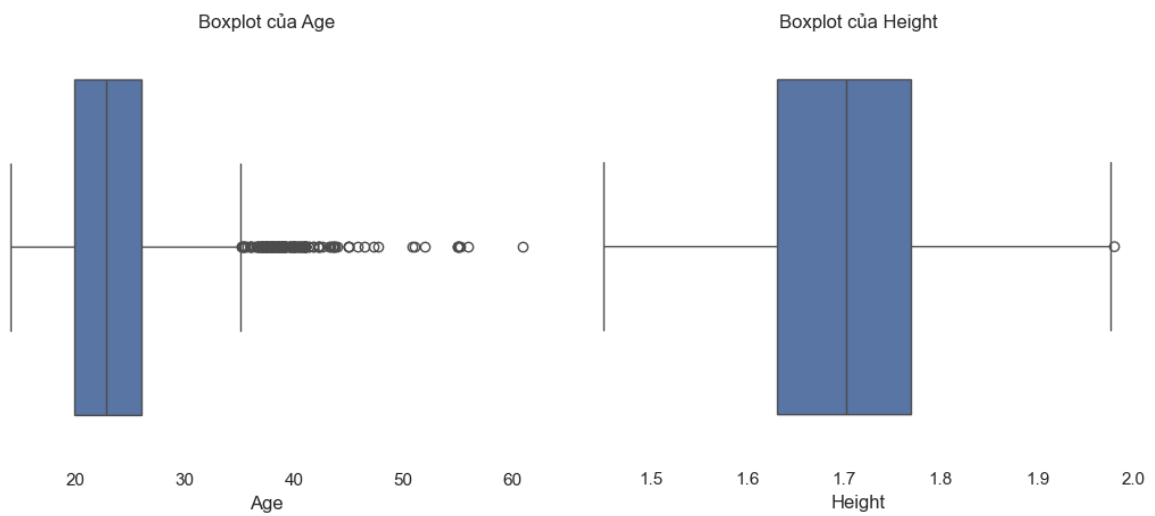
Em vẽ biểu đồ histogram cho tất cả các cột numerical.

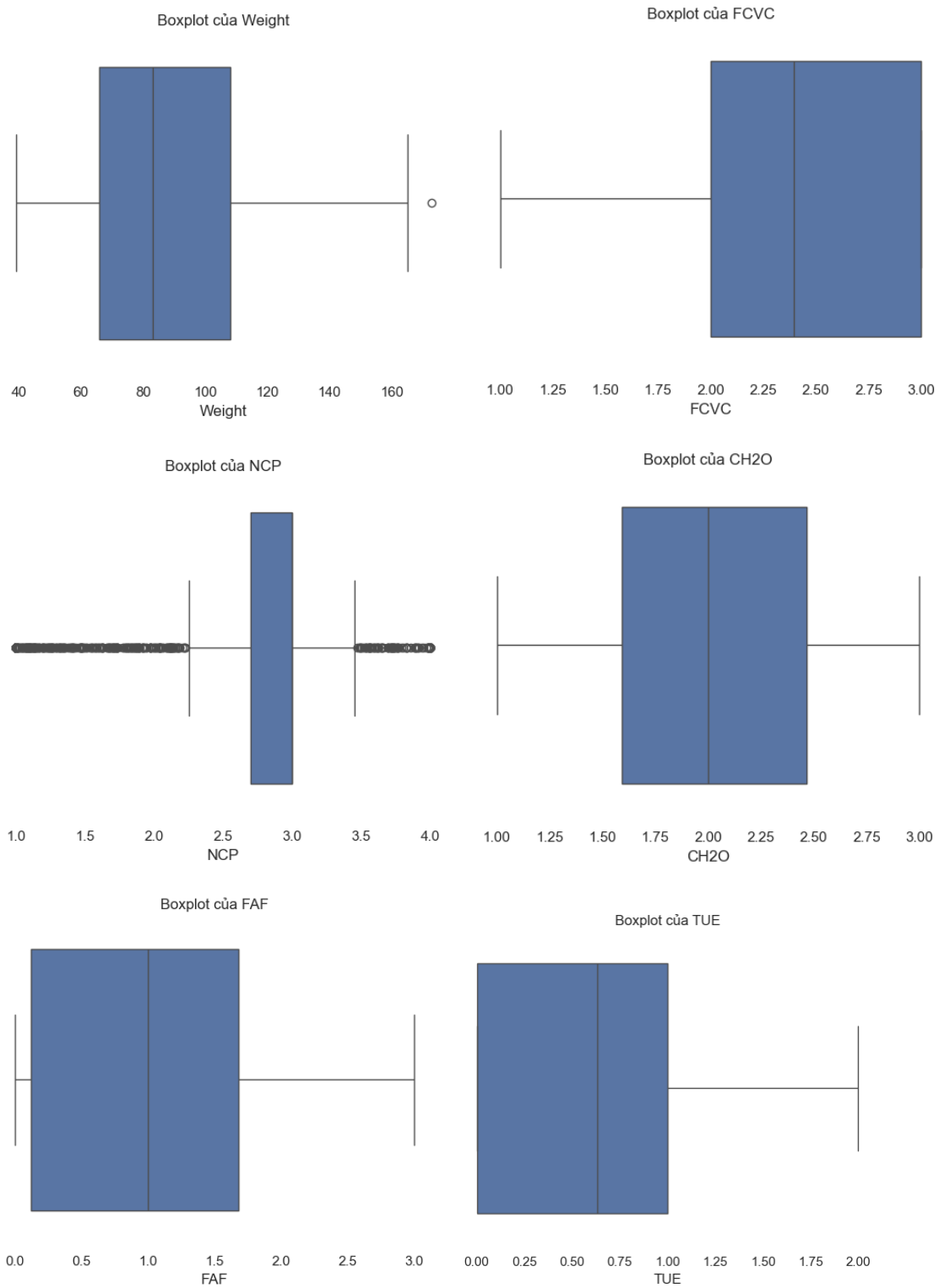




Hình 2.1: Histogram của các cột numerical

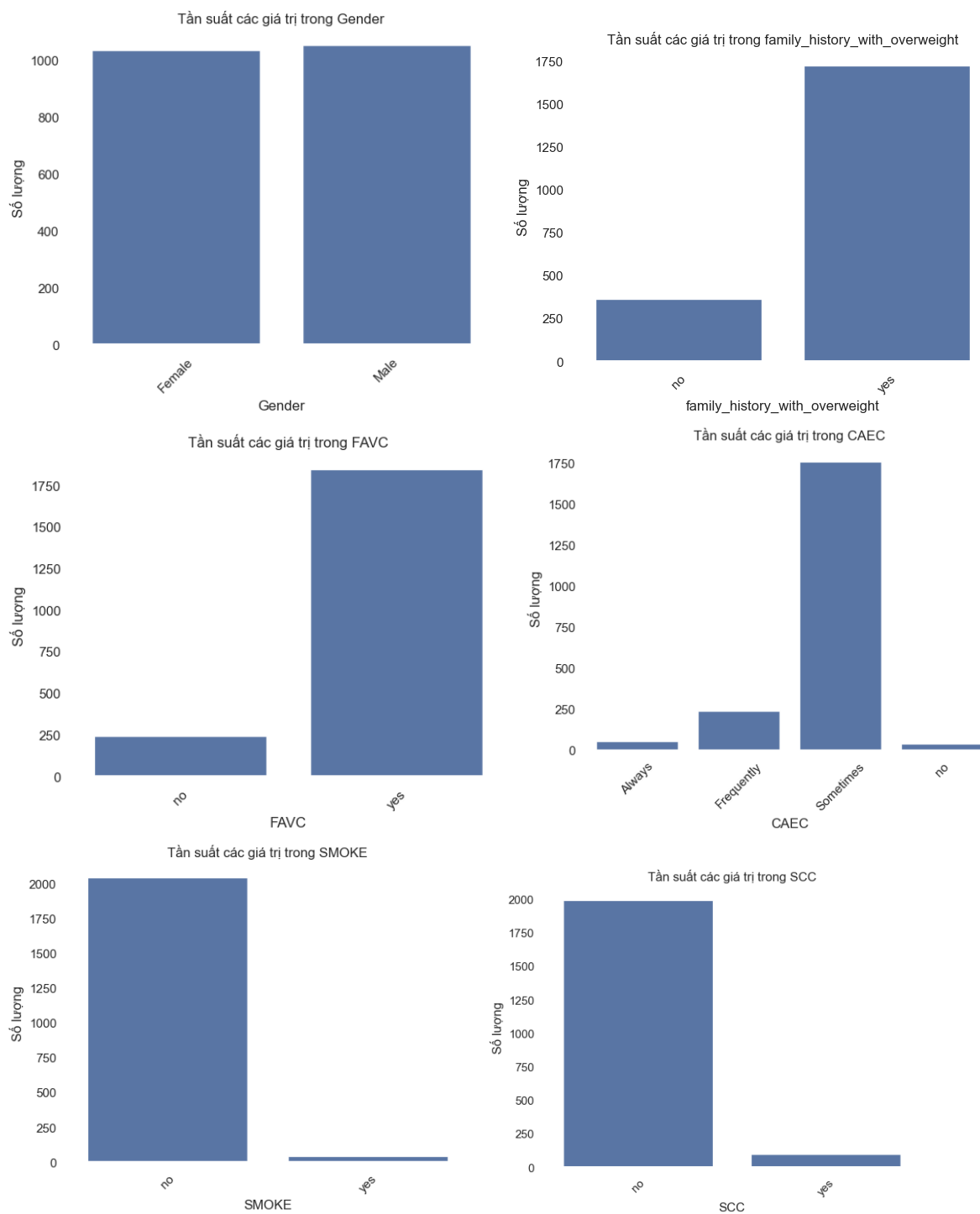
Em vẽ biểu đồ boxplot cho tất cả các cột numerical.

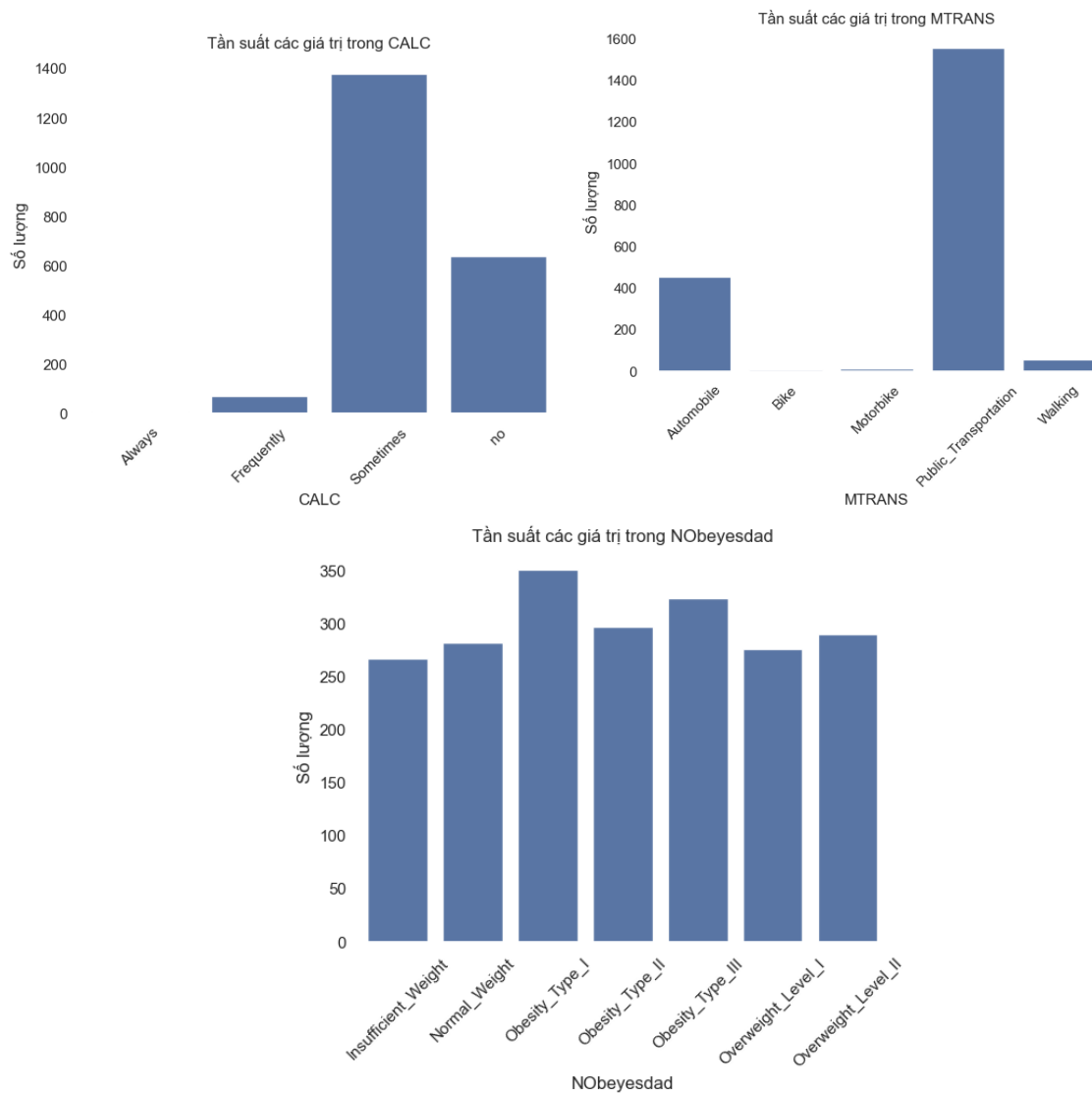




Hình 2.2: Boxplot cho tất cả các cột numerical

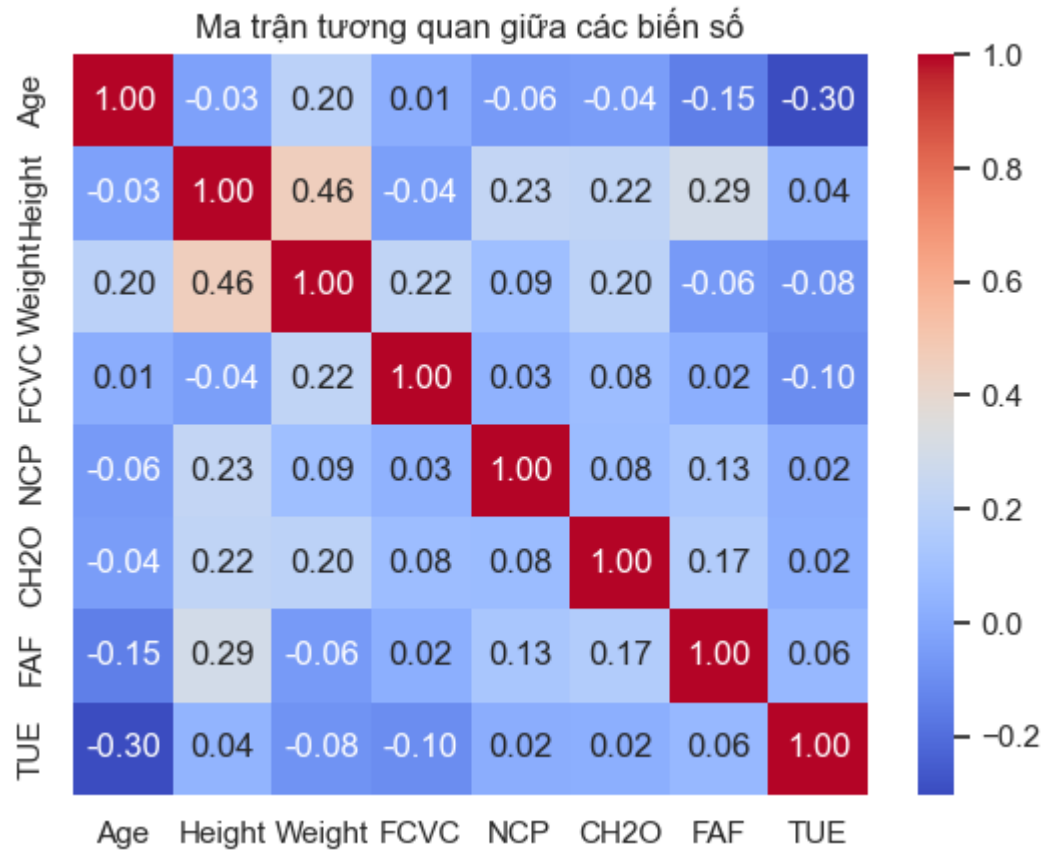
Đối với các cột category em đếm tần suất của các giá trị trong các cột đó.





Hình 2.3: tần suất các giá trị của các cột categorical

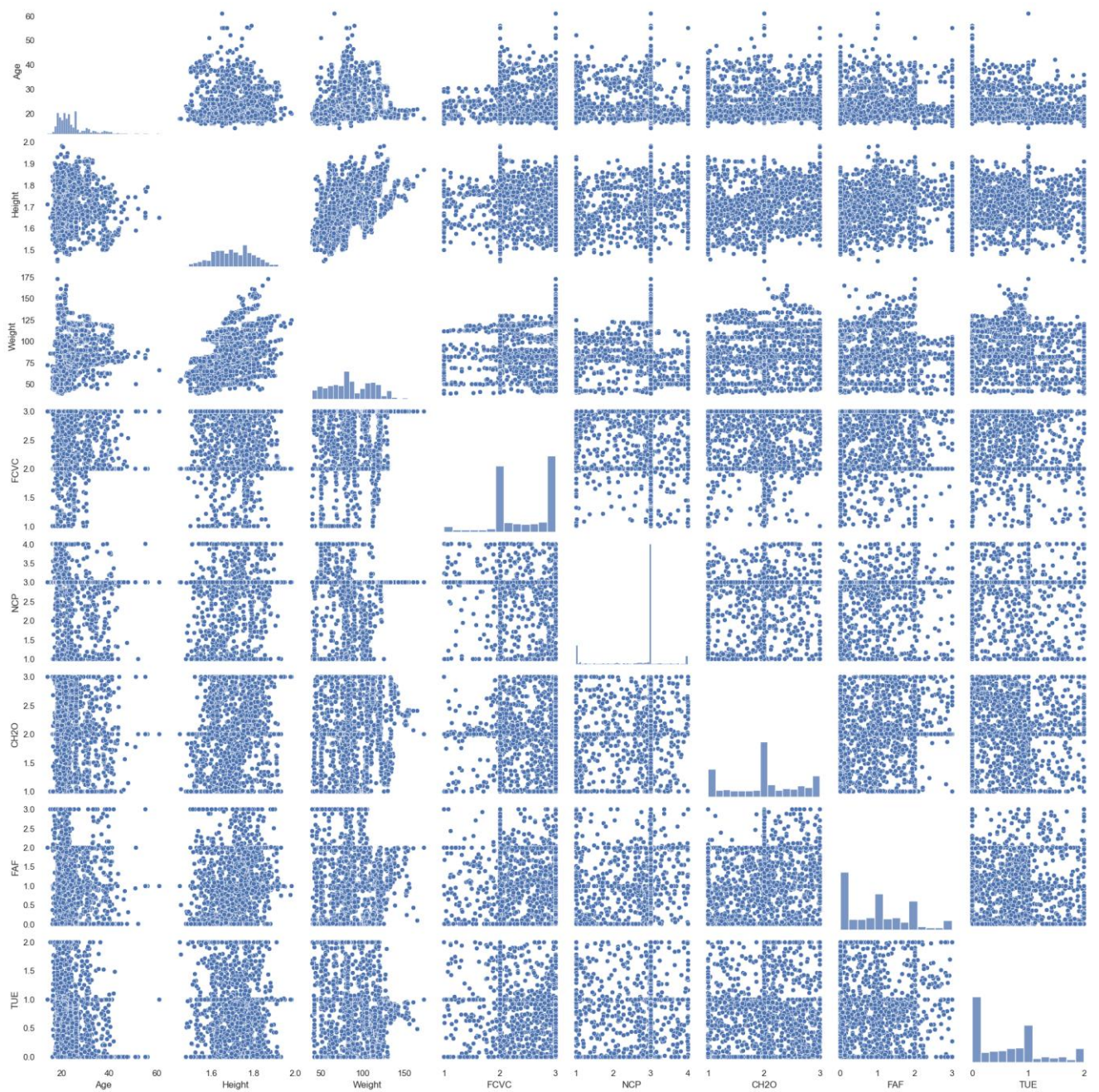
Em vẽ thêm ma trận tương quan giữa các cột numerical



Hình 2.4: Ma trận tương quan giữa các biến numerical

Em vẽ biểu đồ Pairplot cho các cột numerical

Pairplot các biến số



Hình 2.5: Biểu đồ pairplot cho tất cả các cột numerical

2.2 Chuẩn hóa dữ liệu

Trong bước này, ta thực hiện theo pipeline sau:

- **Xác định các biến mục tiêu và các đặc trưng:**
 - Cột 'NObeyesdad' được chọn là `target_col`
 - Dữ liệu của biến target được lưu vào `y`
 - Các cột còn lại (ngoại trừ 'NObeyesdad') được sử dụng làm các biến đặc trưng và lưu vào `X`.
- **Mã hóa dữ liệu:**
 - Các biến đặc trưng trong `X` được chuyển đổi thành dạng số thông qua hàm `pd.get_dummies()`, sử dụng one-hot encoding. Việc sử dụng tham số `drop_first=True` giúp loại bỏ một cột trong mỗi nhóm biến, tránh hiện tượng đa cộng tuyến (dummy variable trap).
 - Biến mục tiêu `y` được chuyển đổi thành các giá trị số thông qua `LabelEncoder()`, giúp dễ dàng xử lý trong quá trình huấn luyện mô hình.
- **Chia dữ liệu thành Train và Test set:**
 - Dữ liệu đã mã hóa được chia thành tập huấn luyện và tập kiểm tra với tỷ lệ 80:20 thông qua hàm `train_test_split()`. Tham số `random_state=42` được sử dụng để đảm bảo tính tái lập của quá trình chia tách.
- **Chuẩn hóa dữ liệu:**
 - Các dữ liệu đặc trưng của tập huấn luyện được chuẩn hóa bằng `StandardScaler()` qua phương pháp `fit_transform()`, giúp đưa các giá trị về cùng một thang đo.
 - Tập kiểm tra sau đó được chuẩn hóa bằng `scaler.transform()`, sử dụng thông số đã tính từ tập huấn luyện để đảm bảo tính nhất quán

2.3 Trình bày và thực hiện các thuật toán machine learning

Trong phần này em sử dụng các thuật toán như Logistic Regression, Decision Tree, Random Forest, SVM, KNN của thư viện scikit-learn và sử dụng các độ đo như Accuracy, Precision, Recall, F1-score, confusion matrix cho từng thuật toán.

2.3.1 Logistic Regression

Logistic Regression là một phương pháp thống kê dùng để dự đoán xác suất xảy ra của một biến phụ thuộc dưới dạng nhị phân (ví dụ: đúng/sai, có/không). Mục tiêu chính của Logistic Regression là ước lượng xác suất mà biến phụ thuộc nhận giá trị 1 (hoặc một lớp cụ thể) dựa trên các biến độc lập. Thay vì dự đoán trực tiếp giá trị số như trong hồi quy tuyến tính, Logistic Regression dự đoán xác suất thuộc về một lớp.

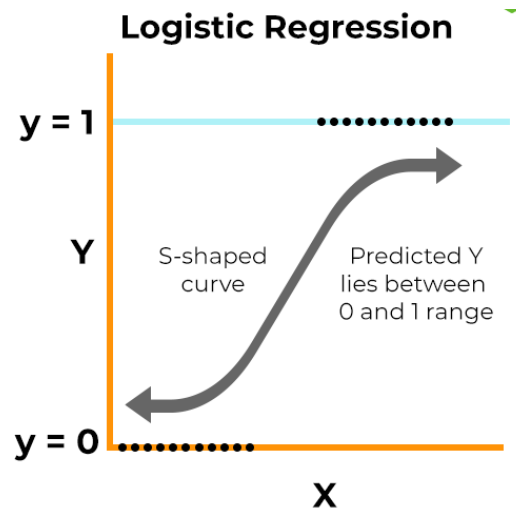
Hàm Logistic (sigmoid)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

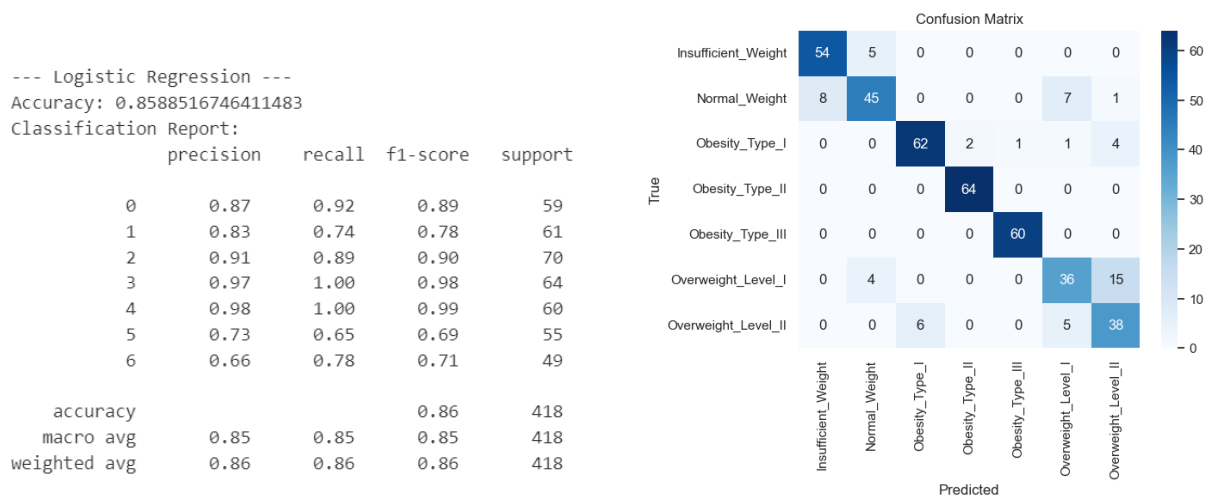
Trong đó $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$. Hàm này giúp đảm bảo đầu ra luôn nằm trong khoảng $[0,1]$.

Các hệ số β trong mô hình được ước lượng thông qua phương pháp Maximum Likelihood Estimation (MLE). Quá trình này tìm ra các giá trị tham số sao cho khả năng xuất hiện của dữ liệu quan sát được là cao nhất.

Sau khi tính được xác suất thông qua hàm sigmoid, ta cần chọn ngưỡng (threshold) để quyết định phân lớp. Thông thường, nếu xác suất ≥ 0.5 , mô hình sẽ dự đoán nhãn 1 (hoặc lớp cụ thể); nếu nhỏ hơn 0.5, dự đoán nhãn 0.



Hình 2.6: Logistic Regression



Hình 2.7: metrics và confusion matrix Logistic Regression

Khi dùng Logistic Regression của thư viện scikit-learn, nó sẽ mặc định sử dụng softmax regression (gọi là “multinomial” loss) để tính xác suất và tối ưu tham số.

2.3.2 Softmax Regression

Softmax Regression (hay còn gọi là Multinomial Logistic Regression) là sự mở rộng của Logistic Regression cho bài toán phân loại đa lớp, tức là khi có hơn hai lớp cần dự đoán. Các điểm khác biệt chính so với Logistic Regression bao gồm:

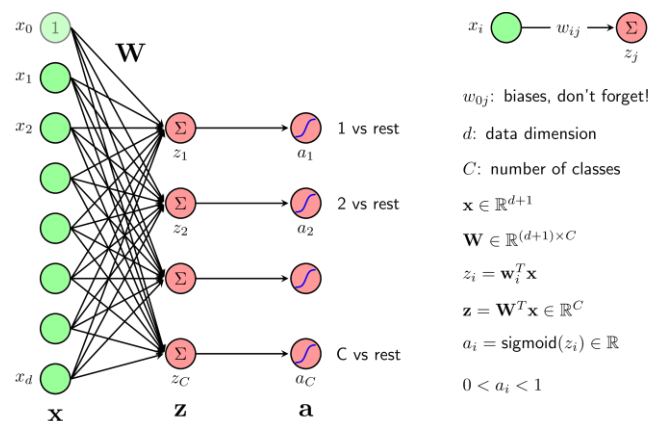
Mục tiêu: Ước lượng xác suất cho mỗi lớp của biến phụ thuộc khi có K lớp. Mỗi lớp k được gán một xác suất $P(y = k|x)$ sao cho tổng các xác suất của tất cả các lớp bằng 1.

Hàm Softmax: Thay vì sử dụng hàm sigmoid cho từng trường hợp nhị phân, softmax sử dụng hàm softmax để chuyển đổi các giá trị tuyến tính z_j thành phân phối xác suất. Công thức của hàm softmax là:

$$P(y = j|x) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

với $z_j = \beta_{0,j} + \beta_{1,j}x_1 + \dots + \beta_{n,j}x_n$ với $j = 1, 2, \dots, K$.

Hàm softmax đảm bảo rằng tổng các xác suất của tất cả các lớp bằng 1.



Hình 2.7: Softmax Regression

(Nguồn internet)

2.3.3 Decision Tree

Decision Tree (Cây quyết định) là một thuật toán học máy có giám sát được sử dụng phổ biến cho cả bài toán phân loại và hồi quy. Nó mô hình hóa quyết định theo dạng cây với các nút (nodes) và cành (branches) để biểu diễn quá trình ra quyết định theo các đặc trưng (features) của dữ liệu.

Cơ chế hoạt động:

- **Nút gốc (Root Node):** Đây là điểm bắt đầu của cây, nơi toàn bộ dữ liệu được đưa vào và sau đó được chia tách dựa trên giá trị của một đặc trưng nhất định.
- **Nút nội (Internal Nodes):** Mỗi nút nội đại diện cho một điều kiện kiểm tra trên một đặc trưng (ví dụ: "Tuổi > 30?"). Từ đó, dữ liệu sẽ được chia thành các nhóm nhỏ hơn theo kết quả của điều kiện.
- **Nút lá (Leaf Nodes):** Đây là các nút cuối cùng của cây, đại diện cho quyết định hoặc dự đoán (ví dụ: phân loại là "Có bệnh" hay "Không có bệnh" trong bài toán phân loại, hoặc giá trị dự đoán trong bài toán hồi quy).

Cách xây dựng:

- **Chọn đặc trưng:** Tại mỗi nút, thuật toán sẽ chọn đặc trưng tốt nhất để chia tách dữ liệu. Các tiêu chí như **Information Gain** (lợi ích thông tin) hoặc **Gini Index** (chỉ số Gini) thường được sử dụng để đánh giá mức độ "sạch" của các nhóm sau khi chia tách.
- **Phân chia dữ liệu:** Dữ liệu được chia tách theo giá trị của đặc trưng được chọn. Quá trình này lặp đi lặp lại đệ quy cho đến khi đạt được điều kiện dừng (ví dụ: cây đạt độ sâu tối đa, số mẫu trong nút nhỏ hơn một ngưỡng nào đó, hoặc không còn sự cải thiện nào trong việc phân chia).

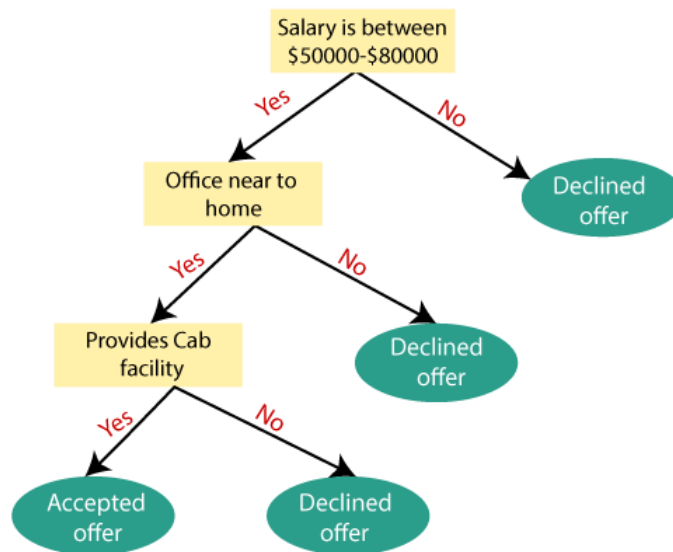
- **Dự đoán:** Khi có một mẫu mới cần dự đoán, nó sẽ được đi qua cây từ nút gốc đến nút lá dựa trên các điều kiện đã được xây dựng, và giá trị của nút lá cuối cùng sẽ là dự đoán của cây.

Ưu điểm:

- **Dễ hiểu và giải thích:** Cấu trúc cây giúp người dùng dễ dàng theo dõi quá trình ra quyết định.
- **Không yêu cầu chuẩn hóa dữ liệu:** Decision Tree có thể xử lý dữ liệu dạng số và dữ liệu dạng phân loại mà không cần chuẩn hóa.
- **Xử lý dữ liệu phi tuyến tính:** Có khả năng xử lý các mối quan hệ phức tạp giữa các đặc trưng.

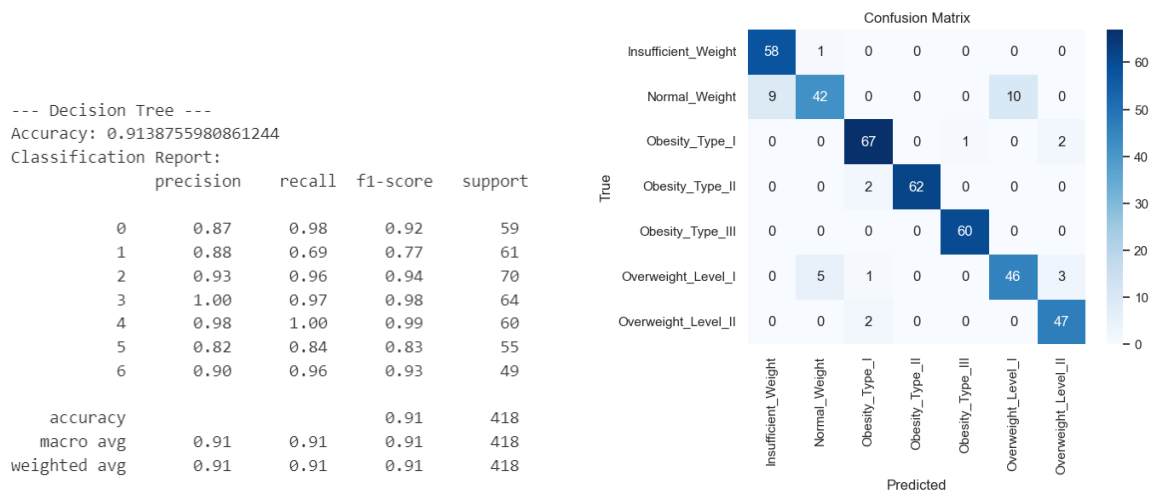
Nhược điểm:

- **Dễ bị overfitting:** Cây quyết định có thể quá khớp với dữ liệu huấn luyện nếu không được kiểm soát độ sâu hoặc sử dụng các kỹ thuật cắt tỉa cây.
- **Sử dụng một số ít đặc trưng:** Một số thuật toán cây có thể ưu tiên chọn ra một số đặc trưng quá mức, bỏ qua các đặc trưng khác.
- **Nhạy cảm với dữ liệu nhiễu:** Các dữ liệu nhiễu có thể làm thay đổi cấu trúc cây và ảnh hưởng đến kết quả dự đoán.



Hình 2.8: Decision Tree

(Nguồn: internet)



Hình 2.9: metrics và confusion matrix Decision Tree

2.3.4 Random Forest

Ensemble (học kết hợp) là một kỹ thuật trong học máy, trong đó kết hợp nhiều mô hình riêng lẻ (gọi là thành phần cơ bản hoặc weak learners) để tạo ra một mô hình mạnh mẽ hơn. Các mô hình kết hợp thường có khả năng cải thiện độ chính xác, giảm thiểu lỗi và tăng cường khả năng tổng quát hóa so với việc sử dụng một mô hình đơn lẻ.

Random Forest là một kỹ thuật học máy theo phương pháp ensemble, được xây dựng dựa trên nhiều cây quyết định (Decision Trees) để cải thiện hiệu suất và độ ổn định của mô hình.

Nguyên lý hoạt động:

- **Ensemble Learning:** Random Forest kết hợp kết quả của nhiều cây quyết định để đưa ra dự đoán cuối cùng. Ý tưởng là nếu có nhiều "cái nhìn" độc lập về dữ liệu, kết quả tổng hợp sẽ chính xác và ổn định hơn.
- **Bootstrap Aggregating (Bagging):** Mỗi cây trong rừng được huấn luyện trên một mẫu con được chọn ngẫu nhiên từ dữ liệu gốc (có phép lặp lại). Quá trình này giúp giảm phương sai và tránh overfitting so với một cây quyết định đơn lẻ.
- **Tính ngẫu nhiên trong lựa chọn đặc trưng:** Khi xây dựng từng cây, tại mỗi nút, Random Forest chỉ chọn ngẫu nhiên một tập con các đặc trưng để tìm kiếm đặc trưng tối ưu chia tách. Điều này giúp các cây trong rừng trở nên đa dạng hơn và không đồng nhất với nhau.

Cách xây dựng:

- Lấy mẫu dữ liệu: Với kỹ thuật bagging, tạo ra nhiều tập dữ liệu huấn luyện khác nhau từ dữ liệu ban đầu.
- Huấn luyện cây: Mỗi cây được xây dựng dựa trên một tập dữ liệu lấy mẫu và tại mỗi nút, chỉ xem xét một tập con đặc trưng ngẫu nhiên.
- Tạo nhiều cây: Quá trình này lặp lại cho đến khi tạo ra một số lượng cây nhất định.
- Phân loại: Mỗi cây trong rừng sẽ đưa ra dự đoán lớp riêng, và lớp xuất hiện nhiều nhất (theo nguyên tắc bỏ phiếu đa số) sẽ là dự đoán cuối cùng.
- Hồi quy: Kết quả dự đoán của các cây sẽ được trung bình lại để cho ra kết quả cuối cùng.

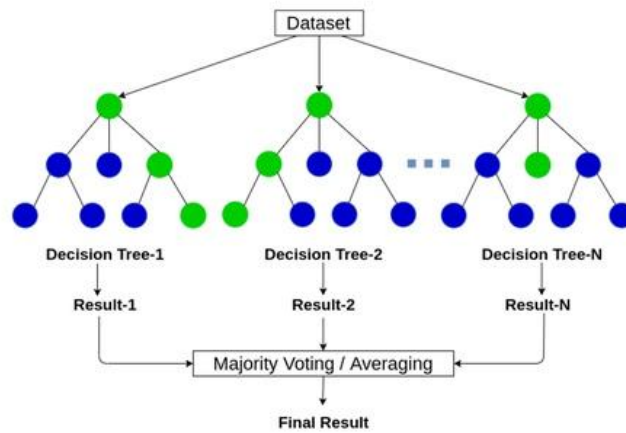
Ưu điểm:

- **Độ chính xác cao:** Bằng cách kết hợp nhiều cây, Random Forest thường cho kết quả dự đoán chính xác hơn.
- **Giảm overfitting:** Sự đa dạng của các cây giúp hạn chế hiện tượng quá khớp (overfitting) với dữ liệu huấn luyện.
- **Đánh giá tính quan trọng của đặc trưng:** Random Forest có thể cung cấp thông tin về mức độ quan trọng của từng đặc trưng trong quá trình dự đoán.
- **Khả năng xử lý dữ liệu lớn và phức tạp:** Mô hình có thể xử lý cả dữ liệu dạng số và dữ liệu dạng phân loại.

Nhược điểm:

- **Khó giải thích:** Do sử dụng nhiều cây, việc diễn giải kết quả và quá trình ra quyết định của mô hình có thể trở nên phức tạp.
- **Yêu cầu tính toán cao:** Việc huấn luyện và dự đoán của nhiều cây có thể tiêu tốn nhiều tài nguyên tính toán và thời gian.
- **Kích thước mô hình lớn:** Do số lượng cây trong rừng có thể rất lớn, nên bộ nhớ cần thiết cũng tăng lên.

Random Forest

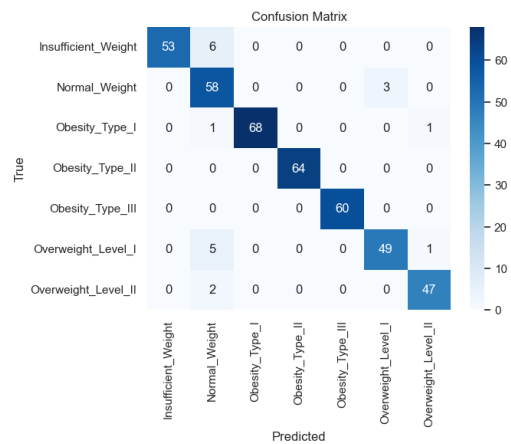


Hình 2.10: Random Forest

(Nguồn internet)

--- Random Forest ---
 Accuracy: 0.9545454545454546
 Classification Report:

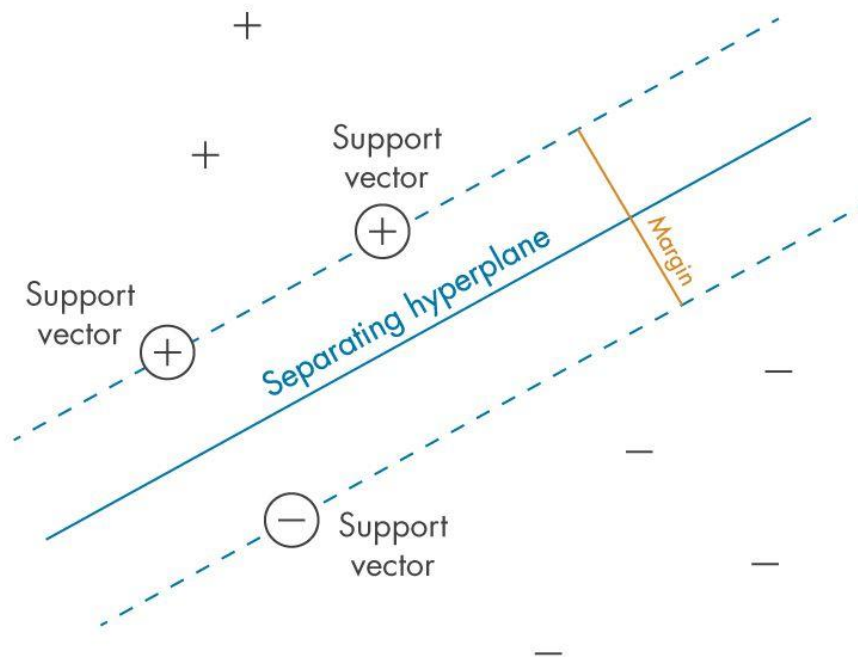
	precision	recall	f1-score	support
0	1.00	0.90	0.95	59
1	0.81	0.95	0.87	61
2	1.00	0.97	0.99	70
3	1.00	1.00	1.00	64
4	1.00	1.00	1.00	60
5	0.94	0.89	0.92	55
6	0.96	0.96	0.96	49
accuracy			0.95	418
macro avg	0.96	0.95	0.95	418
weighted avg	0.96	0.95	0.96	418



Hình 2.11: metrics và confusion matrix Random Forest

2.3.5 Support Vector Machine

Support Vector Machine (SVM) là một thuật toán học máy giám sát được sử dụng chủ yếu cho các bài toán phân loại, và cũng có thể áp dụng cho bài toán hồi quy. SVM cố gắng tìm một siêu phẳng (hyperplane) tối ưu phân chia dữ liệu thành các lớp khác nhau với khoảng cách tối đa giữa các lớp. Siêu phẳng này không chỉ đơn giản là một ranh giới phân chia mà còn tối đa hóa biên (margin) giữa các điểm dữ liệu gần nhất của các lớp, được gọi là support vectors. Support Vectors là những điểm dữ liệu nằm sát ranh giới của các lớp. Các điểm này có ảnh hưởng trực tiếp đến vị trí của siêu phẳng phân chia và do đó là "những người bảo vệ" của biên tối đa.



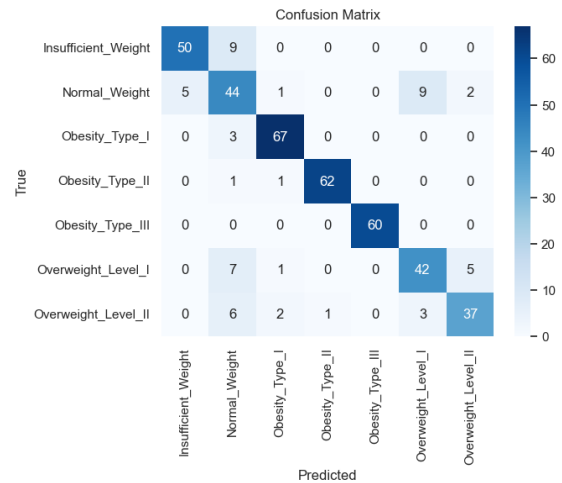
Hình 2.12: Support Vector Machine

--- SVM ---

Accuracy: 0.8660287081339713

Classification Report:

	precision	recall	f1-score	support
0	0.91	0.85	0.88	59
1	0.63	0.72	0.67	61
2	0.93	0.96	0.94	70
3	0.98	0.97	0.98	64
4	1.00	1.00	1.00	60
5	0.78	0.76	0.77	55
6	0.84	0.76	0.80	49
accuracy			0.87	418
macro avg	0.87	0.86	0.86	418
weighted avg	0.87	0.87	0.87	418



Hình 2.13: metrics và confusion matrix SVM

2.3.6 K-Nearest Neighbors

KNN (K-Nearest Neighbors) là một thuật toán học máy giám sát, được sử dụng chủ yếu trong các bài toán phân loại và hồi quy. Đây là một phương pháp "lazy learning", có nghĩa là nó không xây dựng mô hình tổng quát từ dữ liệu huấn luyện trước mà thay vào đó sử dụng toàn bộ dữ liệu huấn luyện để đưa ra dự đoán khi có dữ liệu mới.

Cơ chế hoạt động:

- **Khoảng cách (Distance):** Khi có một điểm dữ liệu mới cần dự đoán, KNN sẽ tính khoảng cách (thường là khoảng cách Euclid hoặc các khoảng cách khác như Manhattan) giữa điểm này và tất cả các điểm trong dữ liệu huấn luyện.
- **Lựa chọn số hàng xóm k (k-neighbors):** Sau khi tính khoảng cách, KNN sẽ chọn ra k điểm gần nhất (k là một số nguyên được chỉ định trước).
- **Quy tắc dự đoán:**
 - **Phân loại:** Dự đoán lớp cho điểm mới dựa trên đa số phiếu của k hàng xóm. Ví dụ: nếu k=5 và 3 trong số 5 hàng xóm thuộc lớp A, thì điểm mới được gán nhãn là lớp A.

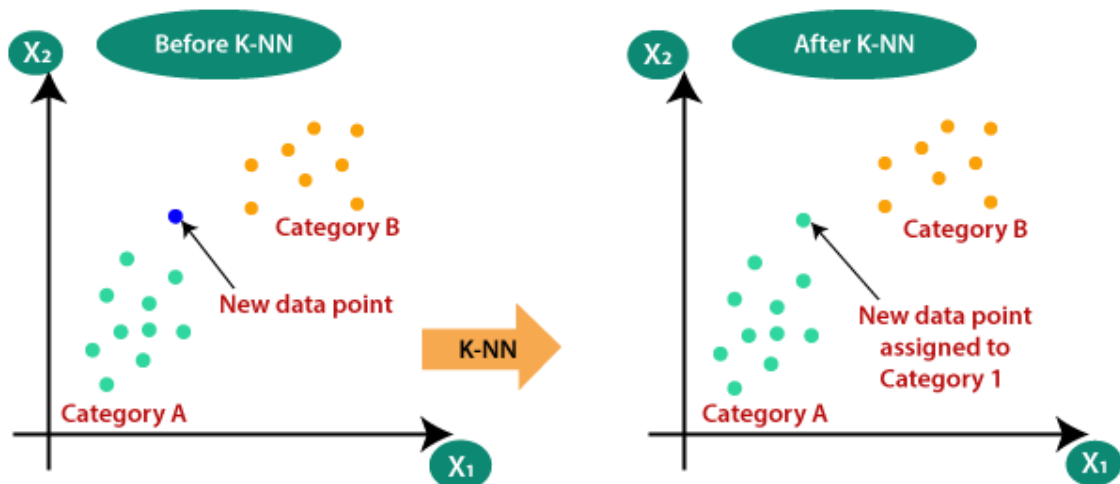
- **Hồi quy:** Giá trị dự đoán là trung bình (hoặc một phép tính khác) của các giá trị đích của k hàng xóm.

Ưu điểm:

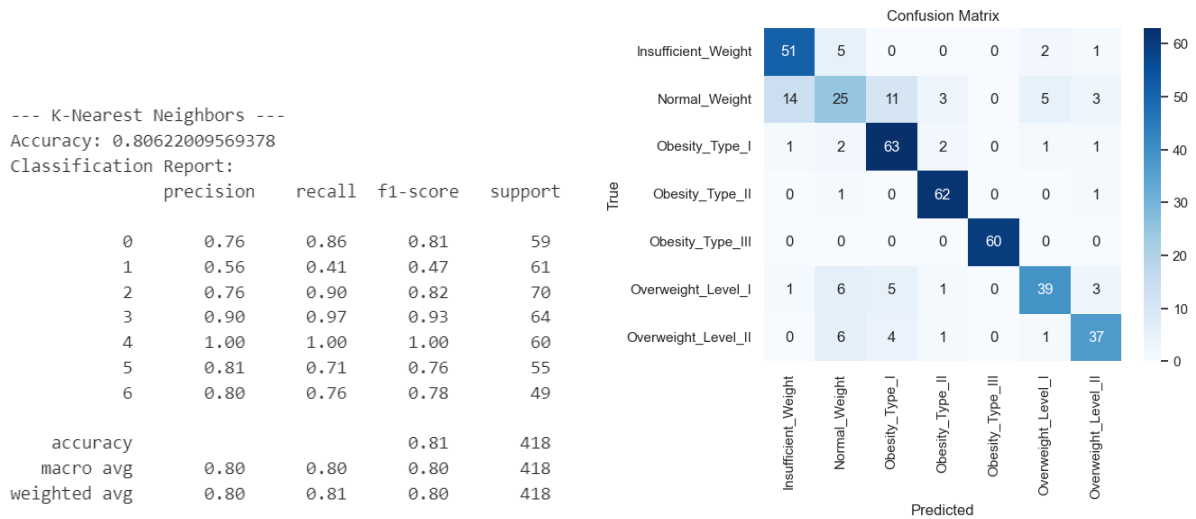
- Dễ cài đặt và hiểu.
- Phù hợp cho cả bài toán phân loại và hồi quy.
- Không giả định về phân phối của dữ liệu, nên có thể áp dụng cho nhiều dạng dữ liệu.

Nhược điểm:

- **Hiệu năng tính toán:** Khi dữ liệu huấn luyện lớn, việc tính khoảng cách với từng điểm có thể chậm.
- **Kích thước dữ liệu:** KNN có thể hoạt động kém với dữ liệu có số chiều (features) rất lớn, do hiện tượng curse of dimensionality.
- **Cân bằng dữ liệu:** Nếu dữ liệu không cân bằng, các lớp có nhiều mẫu có thể chi phối quá mức kết quả dự đoán.



Hình 2.14: K-nearest neighbors



Hình 2.15: metrics và confusion matrix K-nearest neighbors

2.4 Phương pháp, độ đo, và đánh giá kết quả

2.4.1 Bài toán Regression

Đối với bài toán Regression ta thường sử dụng các độ đo như: Mean Squared Error (MSE) và Root Mean Square Error (RMSE), MAE (Mean Absolute Error), R-squared (R^2) ($y_{i,hat}$ là giá trị dự đoán).

- MSE: Tính trung bình phương sai số giữa giá trị dự đoán và giá trị thực

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - Y_{i,hat})^2$$

- Ưu điểm: Phản ánh được độ lệch trung bình của dự đoán.
- Nhược điểm: Đơn vị đo khác với dữ liệu gốc, nhạy cảm với các outlier

- RMSE: Căn bậc hai của MSE

$$RMSE = \sqrt{MSE}$$

- Ưu điểm: Dễ diễn giải và so sánh

- MAE: Tính trung bình trị tuyệt đối của sai số

$$MAE = \frac{1}{n} \sum_{i=1}^n |Y_i - Y_{i,hat}|$$

- Ưu điểm: Ít nhạy cảm với ngoại lệ so với MSE, đơn vị đo giống dữ liệu gốc.
- Nhược điểm: Không phân biệt được lỗi lớn hay nhỏ, vì mỗi sai số đều được coi bằng nhau
- R-squared (R^2): Đo lường tỷ lệ biến thiên của dữ liệu được giải thích của mô hình. (y_{mean} là trung bình của y)

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - y_{i,hat})^2}{\sum_{i=1}^n (y_i - y_{mean})^2}$$

- Ưu điểm: Giúp đánh giá mức độ phù hợp của mô hình (giá trị nằm trong khoảng 0 đến 1).
- Nhược điểm: Có thể gây hiểu lầm nếu sử dụng một cách đơn thuần mà không xét đến các đặc điểm khác của dữ liệu).

2.4.2 Bài toán Classification

Đối với bài toán Classification ta thường sử dụng các độ đo như: Accuracy, Precision, Recall, F1-score, ROC-AUC, Confusion Matrix.

- Accuracy: Tỷ lệ dự đoán đúng trên tổng số mẫu sẵn
 - Ưu điểm: Dễ hiểu, phù hợp với dữ liệu cân bằng
 - Nhược điểm: Có thể gây hiểu lầm khi dữ liệu mất cân bằng.
- Precision: Đo lường tỷ lệ của các trường hợp dự đoán đúng là positive so với tổng số các trường hợp được dự đoán là positive (bao gồm cả các trường hợp TP và FP). Nó là một chỉ số quan trọng để đánh giá khả năng dự đoán đúng của mô hình; phần trăm dự đoán đúng trong các dự đoán là positive. Một giá trị precision cao đồng nghĩa với việc mô hình chỉ đánh giá cao các trường hợp mà nó thực sự tự tin là positive.

$$Precision = \frac{TP}{TP + FP}$$

- Recall: Đo lường tỷ lệ của các trường hợp dự đoán đúng là positive so với tổng số các trường hợp thực tế là positive. Recall là một chỉ số quan trọng để đánh giá khả năng của mô hình trong tất cả các trường hợp positive thực tế. Một giá trị recall cao đồng nghĩa với việc mô hình có khả năng phát hiện nhiều trường hợp positive.

$$Recall = \frac{TP}{TP + FN}$$

- F1-score: Là sự kết hợp giữa Precision và Recall và đây là một phép đo cân bằng giữa Precision và Recall. F1-score thường được sử dụng khi chúng ta muốn có một chỉ số tổng hợp để đánh giá hiệu suất của mô hình mà không ưu tiên quá mức Precision hoặc Recall. F1-score cao chỉ xảy ra khi cả Precision và Recall đều cao.

$$F1 - score = 2 * \frac{Precision * Recall}{Precision + Recall}$$

- ROC curve (Receiver Operating Characteristic Curve): là đồ thị biểu diễn mối quan hệ giữa True Positive Rate và False Positive Rate của mô hình tại các ngưỡng phân loại khác nhau. Khi thay đổi ngưỡng quyết định (threshold), mô hình sẽ cho ra các cặp giá trị (FPR, TPR). Đồ thị ROC thể hiện toàn bộ các cặp giá trị này, giúp ta quan sát được hành vi của mô hình khi thay đổi ngưỡng.
 - True Positive Rate: Recall
 - False Positive Rate: Tỷ lệ mẫu Negative bị dự đoán thành Positive

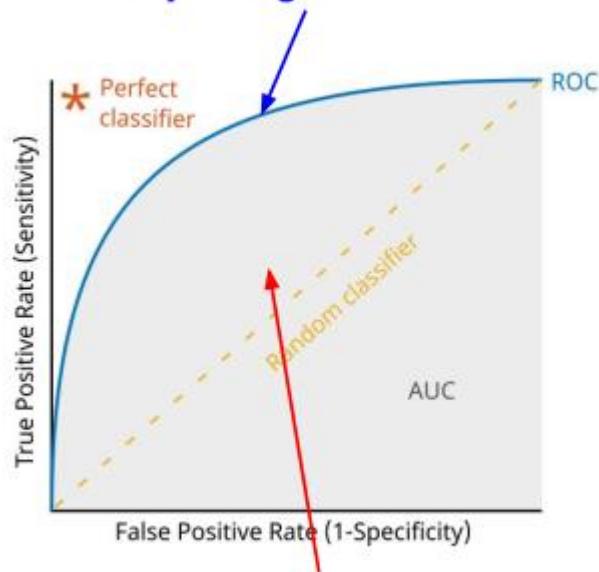
$$FPR = \frac{FP}{FP + TN}$$

- AUC (Area Under the Curve): AUC là đường diện tích dưới ROC, thể hiện khả năng phân biệt giữa các lớp của mô hình

Ý nghĩa:

- Giá trị AUC nằm trong khoảng từ 0 đến 1.
- **AUC = 1**: Mô hình hoàn hảo, có khả năng phân biệt các lớp một cách tuyệt đối.
- **AUC = 0.5**: Mô hình không có khả năng phân biệt, tương đương với việc dự đoán ngẫu nhiên.
- Giá trị AUC càng cao cho thấy mô hình càng tốt trong việc phân biệt giữa lớp dương và âm.

Receiver Operating Characteristic curve



Area under the ROC curve

Hình 2.16: ROC curve

(Nguồn: internet)

- Confusion Matrix: là một bảng tóm tắt kết quả của các dự đoán của mô hình phân loại, so sánh giữa giá trị thực tế và giá trị dự đoán.

Ý nghĩa:

- **Hiểu rõ lỗi của mô hình:** Ma trận nhầm lẫn cho biết rõ ràng mô hình dự đoán sai ở đâu. Ví dụ, nếu FN (sai lầm khi bỏ sót dương tính) chiếm tỷ lệ cao, bạn có thể cần cải thiện Recall.
- **Tùy chỉnh mô hình:** Qua ma trận nhầm lẫn, bạn có thể nhận diện các khu vực mà mô hình còn yếu và từ đó điều chỉnh ngưỡng dự đoán, cân bằng lại dữ liệu hoặc thay đổi kiến trúc mô hình.
- **Đối với bài toán đa lớp:** Ma trận nhầm lẫn được mở rộng thành bảng có kích thước $n \times n$ (với n là số lớp), giúp ta xem xét chi tiết lỗi giữa từng cặp lớp.

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Hình 2.17: Confusion Matrix
(Nguồn: internet)

2.5 Overfitting

2.5.1 Cơ sở lý thuyết và phương pháp

Overfitting (quá khớp) xảy ra khi mô hình học quá kỹ vào dữ liệu huấn luyện, nắm bắt cả nhiễu (noise) và các đặc trưng không tổng quát. Kết quả là, mô hình đạt hiệu năng rất cao trên tập huấn luyện nhưng lại cho kết quả kém khi áp dụng vào dữ liệu mới (tập kiểm tra).

Đặc điểm:

- Hiệu năng trên tập huấn luyện rất tốt, nhưng hiệu năng trên tập kiểm tra (validation/test) kém.
- Sai số huấn luyện giảm dần theo thời gian, trong khi sai số trên tập kiểm tra bắt đầu tăng lên sau một khoảng thời gian nhất định.
- Mô hình phức tạp với nhiều tham số và có khả năng biểu diễn các mối quan hệ quá chặt chẽ với dữ liệu huấn luyện.

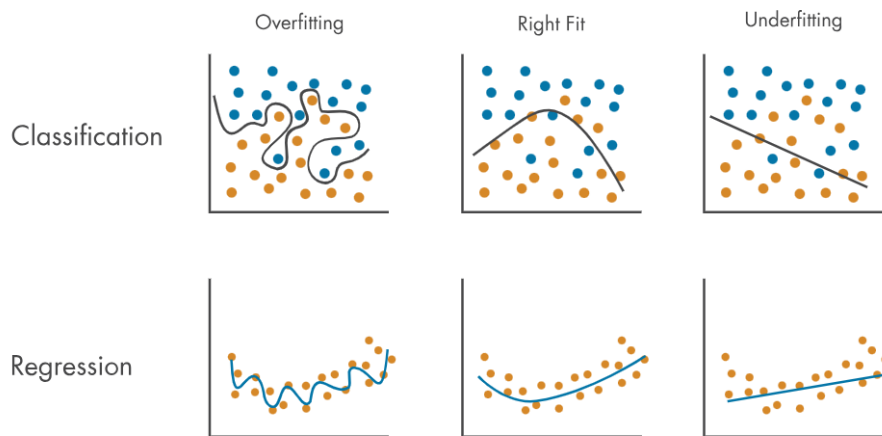
Nguyên nhân gây ra overfitting:

- Mô hình máy học quá phức tạp: khi tập dữ liệu quá nhỏ nhưng chúng ta lại phát triển một mô hình máy học quá lớn với nhiều tham số thì khả năng học dữ liệu của mô hình sẽ tăng quá mức. Do đó nó sẽ “học kỹ” chi tiết dữ liệu mà không phải tổng quát dữ liệu.
- Kích thước dữ liệu không đủ: để huấn luyện mô hình máy học chúng ta cần phải có tập dữ liệu có kích thước tương ứng với độ phức tạp của mô hình, đảm bảo kích thước dữ liệu không nhỏ hơn mô hình máy học. Điều này có thể dẫn đến việc không đủ dữ liệu để mô hình có thể học một cách tổng quát.
- Dữ liệu là nhiễu (noise): một tập dữ liệu khi được huấn luyện có nhiều lỗi hoặc không chính xác, dẫn đến việc học của mô hình máy học bị nhầm lẫn và nhiễu với các tiếng ồn dẫn đến khi dự đoán trên tập dữ liệu sách cho ra kết quả kém
- Mô hình học máy được huấn luyện có phương sai cao nhưng độ lệch thấp, độ chính xác trong quá trình huấn luyện của mô hình tăng lên nhưng độ chính xác thực sẽ giảm.

Các giải pháp giải quyết Overfitting

- Tăng kích thước tập dữ liệu huấn luyện: dữ liệu trong tập huấn luyện quá nhỏ thì khi thực hiện huấn luyện mô hình máy học phức tạp có thể gây ra overfitting.
- Loại bỏ các đặc trưng không cần thiết: nên loại bỏ các đặc trưng không quan trọng để việc học của mô hình tốt hơn, giảm thiểu việc bị nhiễu trong quá trình học trên tập dữ liệu huấn luyện.
- Loại bỏ các outlier
- Giảm độ phức tạp của mô hình: có thể giảm độ phức tạp của mô hình bằng cách đơn giản hóa các tham số.
- Sử dụng Regularization:
 - L1 Regularization (Lasso): Thêm một thuật toán phạt theo tổng trị tuyệt đối của các hệ số (weights). Giúp giảm bớt số lượng đặc trưng không quan trọng, thúc đẩy tính đơn giản cho mô hình.
 - L2 Regularization (Ridge): Thêm một thuật toán phạt theo tổng bình phương của các hệ số. Giúp giảm thiểu giá trị các tham số, tránh trường hợp một số tham số quá lớn.
- Early stopping: Theo dõi hiệu năng trên tập validation trong quá trình huấn luyện. Khi sai số trên tập validation bắt đầu tăng (ngụ ý mô hình bắt đầu overfitting), dừng quá trình huấn luyện để giữ lại mô hình ở trạng thái tốt nhất.
- Cross-Validation: Sử dụng k-fold cross-validation, phương pháp này không chỉ giúp đánh giá hiệu năng của mô hình trên nhiều tập con mà còn giúp điều chỉnh tham số sao cho mô hình không quá khớp với tập huấn luyện cụ thể.
- Ensemble: Kết hợp nhiều mô hình, sử dụng các kỹ thuật ensemble như bagging, boosting hay random forests để tổng hợp dự đoán từ nhiều mô hình, qua đó giảm thiểu nguy cơ một mô hình đơn lẻ overfit.
- Feature Selection và Dimensionality Reduction

- Chọn lọc các đặc trưng quan trọng: Loại bỏ các đặc trưng thừa, không cần thiết hoặc nhiễu, từ đó giảm độ phức tạp của mô hình.
- Kỹ thuật giảm chiều dữ liệu: Sử dụng PCA (Principal Component Analysis) hoặc các kỹ thuật khác để giảm số lượng đặc trưng mà vẫn giữ lại các thông tin chính của dữ liệu.



Hình 2.18: Overfitting
(Nguồn: internet)

2.5.2 So sánh

Để áp dụng các phương pháp giải quyết Overfitting, ở phần áp dụng vào code. Em sử dụng GridSearch cho từng mô hình để lấy các hyper parameter tốt nhất. Kết quả nhận được các hyper parameter sau:

- **Best LR params:** {'C': 10, 'penalty': 'l1', 'solver': 'liblinear'}
- **Best DT params:** {'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 5}
- **Best RF params:** {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 100}
- **Best SVM params:** {'C': 10, 'gamma': 'scale', 'kernel': 'linear'}

- **Best KNN params:** {'metric': 'manhattan', 'n_neighbors': 3, 'weights': 'distance'}

	Model	Baseline Accuracy	Tuned Accuracy
0	Logistic Regression	0.858852	0.777512
1	Decision Tree	0.913876	0.959330
2	Random Forest	0.954545	0.942584
3	SVM	0.866029	0.971292
4	K-Nearest Neighbors	0.806220	0.842105

Hình 2.19: So sánh mô hình

Kết luận:

Trong thí nghiệm này, em đã đánh giá năm thuật toán phân lớp phổ biến: Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM) và K-Nearest Neighbors (KNN). Với mỗi thuật toán, em so sánh độ chính xác (accuracy) trên tập kiểm thử trước và sau khi tinh chỉnh siêu tham số bằng GridSearchCV.

- **Logistic Regression**
 - Việc sử dụng penalty L1 với C=10 đã làm giảm đáng kể độ chính xác so với cấu hình mặc định. L1 regularization đã loại bỏ nhiều hệ số quan trọng, dẫn đến underfitting.
- **Decision Tree**
 - Sau khi tối ưu siêu tham số (criterion, min_samples_split, min_samples_leaf), mô hình cây quyết định đã cải thiện mạnh mẽ. Việc tăng min_samples_split giúp giảm độ phức tạp của cây, loại bỏ các nhánh chia nhỏ quá mức, từ đó cải thiện khả năng khái quát và nâng cao độ chính xác.
- **Random Forest**

- Random Forest vốn đã có tính kháng overfitting cao nhờ bagging và feature subsampling. Khi áp dụng các thông số tối ưu (`n_estimators=100`, `max_features='sqrt'`, `min_samples_split=5`), độ chính xác giảm nhẹ do underfitting, việc tăng `min_samples_split` làm giảm khả năng học các pattern nhỏ. Để cải thiện, có thể tăng số cây (`n_estimators`) hoặc giảm `min_samples_split`.
- **Support vector Machine (SVM)**
 - SVM tuyến tính với `C=10` và `gamma='scale'` cho kết quả rất ấn tượng. Việc chọn kernel tuyến tính phù hợp với dữ liệu đã cho giúp phân tách rõ ràng hai lớp mà không bị overfitting. Đây là một trong những cấu hình thành công nhất trong thử nghiệm.
- **K-Nearest Neighbors (KNN)**
 - Việc tinh chỉnh `n_neighbors`, sử dụng trọng số theo khoảng cách và metric Manhattan đã cải thiện độ chính xác đáng kể. KNN nhạy với scale dữ liệu, vì vậy việc chuẩn hóa trước khi huấn luyện cũng góp phần nâng cao hiệu năng.

Qua thí nghiệm này, em rút ra được trong một vài mô hình machine learning. Việc ta can thiệp các hyper parameter có thể không khiến mô hình có hiệu suất tốt hơn các hyper parameter mặc định. Tuy nhiên, nếu như ta có đủ tài nguyên phần cứng và thời gian, thì ta hoàn toàn có thể tìm ra được các hyper parameter tốt nhất để cho mô hình đạt hiệu suất vượt trội.

TÀI LIỆU THAM KHẢO

Tiếng Việt

[1] Vũ Khắc Tiệp, “Machine Learning cơ bản” series, [Online]. Available: <https://machinelearningcoban.com/>. [Accessed: Apr. 2, 2025].

Tiếng Anh

[1] Andrew Ng, “Machine Learning”, Coursera, Stanford University, [Online]. Available: <https://www.coursera.org/learn/machine-learning>. [Accessed: Dec 1, 2024].

